

Week 8 - Multivariate Analysis

Please run the cells of the notebook as you get to them while reading

```
import pandas as pd
import numpy as np
from datetime import datetime, timedelta
import seaborn as sns
from sklearn import decomposition
from sklearn import linear_model
from sklearn import metrics
```

1. Lesson: Multivariate graphs

In this lesson, we'll consider some multivariate plots: heatmaps and bubble plots. We'll also consider principal component analysis and multiple regression.

Heatmaps

In this plot, we use a heatmap to count the total traffic tickets given to citizens. A heatmap requires the x and y axes to be categorical, while the color involves a third, numerical variable. In this example, each row of the DataFrame represents a traffic ticket given to one person in a particular city and year. The data are given as three columns (city, year, and ticket), so we must use a pivot_table to transform it into a two-dimensional grid. The cmap is set to "Blues" in the first example, meaning that the heatmap uses only blue as its color. However, we could also use a multicolor map like "viridis" in the second example. I personally think "Blues" is easier to read - what do you think? Here is a reference guide to the color maps:

https://matplotlib.org/stable/gallery/color/colormap_reference.html

```
np.random.seed(0)
num_samples = 100
city_series = np.random.choice(["New York", "Boston", "Atlanta",
                                "Washington, D. C."], size = num_samples)
year_series = np.random.choice([2018, 2019, 2020], size = num_samples)
ticket_series = np.random.randint(50, 150, size = num_samples) #
Traffic tickets can be from $50 to $150
df = pd.DataFrame({"city": city_series, "year": year_series, "ticket":
                    ticket_series})
pivot = df.pivot_table(index = ["city"], columns = ["year"], values =
                        "ticket", aggfunc = "sum")

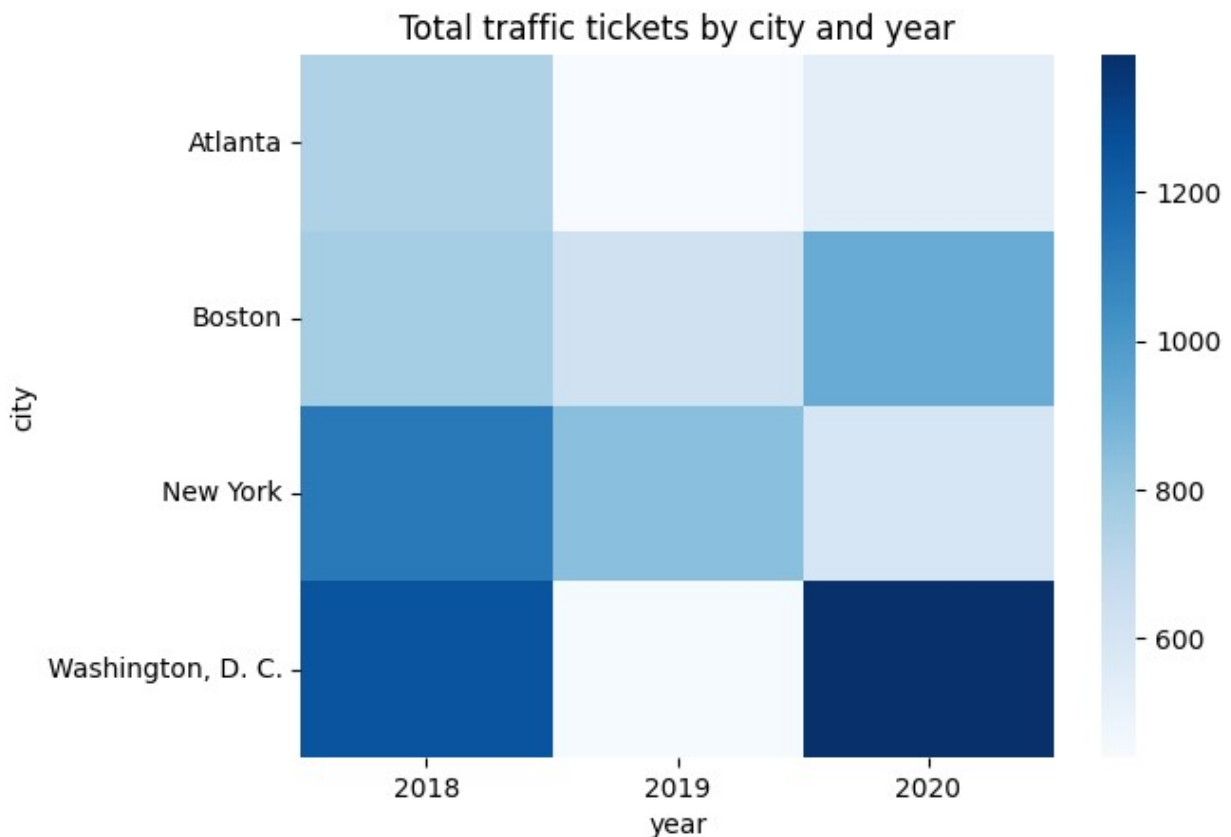
df.iloc[0:5] # first five rows of a long DataFrame
```

	city	year	ticket
0	New York	2019	79
1	Washington, D. C.	2020	71
2	Boston	2018	75
3	New York	2018	130
4	Washington, D. C.	2020	110

pivot # 2 by 2 grid showing total ticket amounts

year	2018	2019	2020
city			
Atlanta	742	440	538
Boston	775	628	922
New York	1122	841	596
Washington, D. C.	1258	451	1385

```
ax = sns.heatmap(pivot, cmap = "Blues")
ax.set_title("Total traffic tickets by city and year")
Text(0.5, 1.0, 'Total traffic tickets by city and year')
```

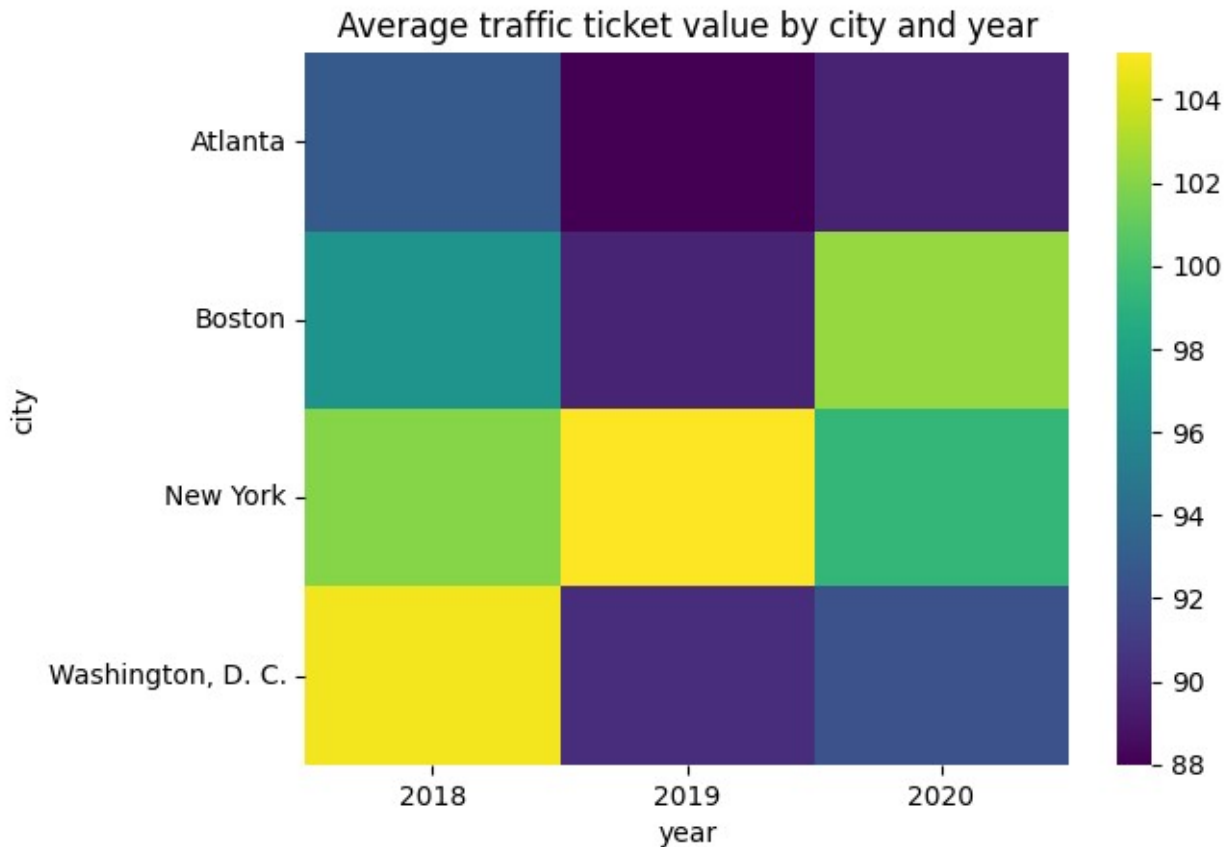


```
pivot2 = df.pivot_table(index = ["city"], columns = ["year"], values =
"ticket", aggfunc = "mean") # note mean, not sum this time
ax2 = sns.heatmap(pivot2, cmap = "viridis") # do you think this is a
```

better or worse scale than Blues?

```
ax2.set_title("Average traffic ticket value by city and year")
```

```
Text(0.5, 1.0, 'Average traffic ticket value by city and year')
```



Bubble plots

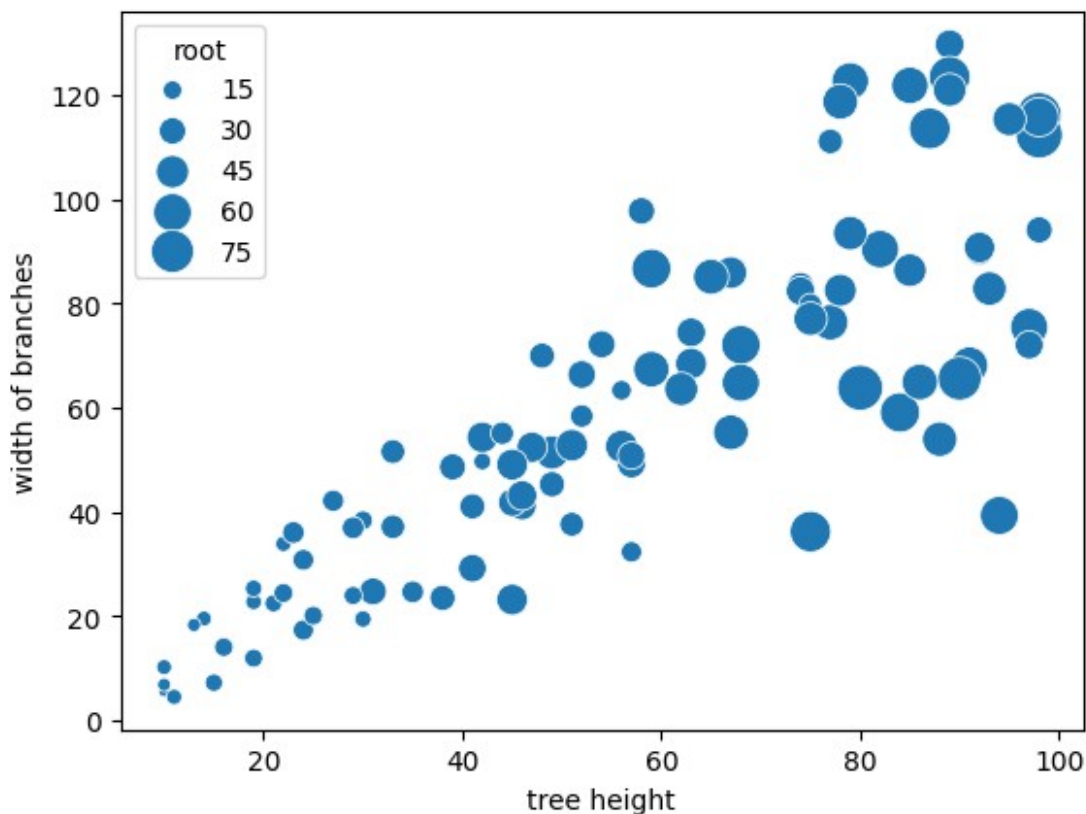
A bubble plot is just a scatter plot, but with the dot size representing a third variable. Here, each data point shows a tree (I mean the kind of tree that is a plant and grows in the ground, not a decision tree), with its height, width of branches, and root depth. The bubble size in the bubble plot represents the root depth, while the other variables are shown on the x and y axes. From this plot, we can see that the root depth is strongly correlated with both the width of branches and the tree height.

```
np.random.seed(0)
num_samples = 100
height_series = np.random.randint(10, 100, size = num_samples) # tree height
width_series = height_series * np.maximum(np.random.normal(scale = 0.3, loc = 1, size = num_samples), 0.25) # strongly related to height
root_series = height_series * np.maximum(np.random.normal(scale = 0.2, loc = 0.7, size = num_samples), 0.3) # strongly related to height
df2 = pd.DataFrame({'height': height_series, 'width': width_series,
```

```

'root': root_series})
minroot = root_series.min()
maxroot = root_series.max()
maxbubble = 300
ax = sns.scatterplot(data = df2, x = "height", y = "width", size =
"root", sizes = (minroot / maxroot * maxbubble, maxbubble))
ax.set_xlabel("tree height")
ax.set_ylabel("width of branches")
Text(0, 0.5, 'width of branches')

```



Principal Component Analysis

Principal Component Analysis can be thought of as a way to use one (or a few) linear combinations of the coordinates in order to explain all of the coordinates. It is most useful when there are strong correlations between some or all of the coordinates. Principal Component Analysis can show us which linear combinations are most (and least) useful for explaining the data.

With the trees data, the three values (height, width, and root) are highly correlated. For this reason, Principal Component Analysis shows that most of the variance can be explained by a single value which is a linear combination of height, width, and root. What this means is that if the first components_ vector in the code below (the "first principal component") is $[a, b, c]$, for some specific numbers a , b , and c , then the value $v = a * (\text{height} - \mu_h) + b * (\text{width} - \mu_w) + c$

* (root - mu_r), where the mu's are the means, is highly informative about the values of height, width, and root. In this case, because height, width, and root are so highly correlated, we can almost assume that all of the data is located along an axis defined by the vector [a, b, c]. The point $v * [a, b, c] + (\text{mean vector}) = (a * (\text{height} - \text{mu_h}) + b * (\text{width} - \text{mu_w}) + c * (\text{root} - \text{mu_r})) * [a, b, c] + (\text{mean vector})$ is therefore approximately equal to the data point (height, width, root) in question. The mean vector is [mu_h, mu_w, mu_r]. Importantly, there is still variance unexplained by this one value - just knowing $v * [a, b, c]$ does not tell you the height, width, and root with 100% accuracy. If the second principal component is [d, e, f], then we could approximate the tree vector even more accurately as: $(\text{height}, \text{width}, \text{root}) = (a * (\text{height} - \text{mu_h}) + b * (\text{width} - \text{mu_w}) + c * (\text{root} - \text{mu_r})) * [a, b, c] + (d * (\text{height} - \text{mu_h}) + e * (\text{width} - \text{mu_w}) + f * (\text{root} - \text{mu_r})) * [d, e, f] + (\text{mean vector})$.

The printing of "components_", below, shows the first principal component [a, b, c]. If you draw a vector in the direction shown, you will be heading in the direction where the values height, width, and root change most rapidly. A later computation shows all three principal component vectors.

Optional exercise for the reader: show that if all three principal components are included, then equality is exact. This is going to be true simply because the principal components are orthonormal: the projections onto a complete set of orthonormal vectors always add up to the original vector.

$(\text{height}, \text{width}, \text{root}) = (a * (\text{height} - \text{mu_h}) + b * (\text{width} - \text{mu_w}) + c * (\text{root} - \text{mu_r})) * [a, b, c] + (d * (\text{height} - \text{mu_h}) + e * (\text{width} - \text{mu_w}) + f * (\text{root} - \text{mu_r})) * [d, e, f] + (g * (\text{height} - \text{mu_h}) + h * (\text{width} - \text{mu_w}) + i * (\text{root} - \text{mu_r})) * [g, h, i] + [\text{mu_h}, \text{mu_w}, \text{mu_r}]$.

For this proof, we must use the fact that the principal component vectors are orthonormal. That is, any pair of them are orthogonal, even with a transpose ($ad + be + cf = 0$, $ag + bh + ci = 0$, $dg + eh + fi = 0$, $ab + de + gh = 0$, $ac + df + gi = 0$, $bc + ef + hi = 0$) and any one of them is a unit vector, even with a transpose ($a^2 + b^2 + c^2 = 1$, $d^2 + e^2 + f^2 = 1$, $g^2 + h^2 + i^2 = 1$, $a^2 + d^2 + g^2 = 1$, $b^2 + e^2 + h^2 = 1$, $c^2 + f^2 + i^2 = 1$). If you write out the equation above as a single vector, then apply the orthonormality rules, you should be able to prove equality.

```
pca = decomposition.PCA(n_components = 1)
pca.fit(df2)
print(f"The first principal component explains
{pca.explained_variance_ratio_[0]} of the variance")
print(f"The components_ vector is [a, b, c] = {pca.components_}")
mu_h, mu_w, mu_r = df2.mean().values
print(f"(height, width, root) is approximately equal to
({pca.components_[0][0]:.2} * (height - {mu_h:.2}) +
{pca.components_[0][1]:.2} * (width - {mu_w:.2}) + {pca.components_[0][2]:.2} * (root - {mu_r:.2})) * ({pca.components_[0][0]:.2},
{pca.components_[0][1]:.2}, {pca.components_[0][2]:.2}) + ({mu_h:.2},
{mu_w:.2}, {mu_r:.2})")
print("Examples: (estimating the the vector equals its projection onto
the first principal component)")
df_example = df2.copy()
df_example["v"] = pca.components_[0][0] * (df2["height"] - mu_h) +
pca.components_[0][1] * (df2["width"] - mu_w) + pca.components_[0][2]
```

```

* (df2["root"] - mu_r)
df_example["height_est"] = df_example["v"] * pca.components_[0][0] +
mu_h
df_example["width_est"] = df_example["v"] * pca.components_[0][1] +
mu_w
df_example["root_est"] = df_example["v"] * pca.components_[0][2] +
mu_r
print(df_example.iloc[0:5])
print("Means:")
print(f"mu_h = {mu_h:.3}", f"mu_w = {mu_w:.3}", f"mu_r = {mu_r:.3}")

```

The first principal component explains 0.8686108480944689 of the variance

The components_ vector is [a, b, c] = [[0.59031629 0.71908728 0.36666083]]

(height, width, root) is approximately equal to $(0.59 * (\text{height} - 5.6e+01) + 0.72 * (\text{width} - 5.9e+01) + 0.37 * (\text{root} - 3.6e+01)) * (0.59, 0.72, 0.37) + (5.6e+01, 5.9e+01, 3.6e+01)$

Examples: (estimating the the vector equals its projection onto the first principal component)

	height	width	root	v	height_est	width_est	root_est
0	54	72.139334	31.517932	6.287751	60.001762	63.563941	38.673095
1	57	49.169527	33.768693	-7.633330	51.783921	53.553468	33.568780
2	74	83.443733	28.770850	25.215680	71.175227	77.174773	45.613225
3	77	76.353866	53.061241	30.794730	74.468631	81.186597	47.658844
4	77	111.095365	26.006307	45.856916	83.360084	92.017624	53.181557

Means:

mu_h = 56.3 mu_w = 59.0 mu_r = 36.4

If we were to choose a second or third component to preserve, that values below show that each would explain a small, additional amount of the variance. This means that with this particular dataset, we should likely do PCA using only one component and not two. If the components' explained variances were closer in magnitude to each other, we might have to include two or even all three values to explain the data.

```

pca3 = decomposition.PCA(n_components = 3)
pca3.fit(df2)
print(pca3.explained_variance_ratio_)
print(pca3.components_)

[0.86861085 0.09414617 0.03724298]
[[ 0.59031629  0.71908728  0.36666083]

```

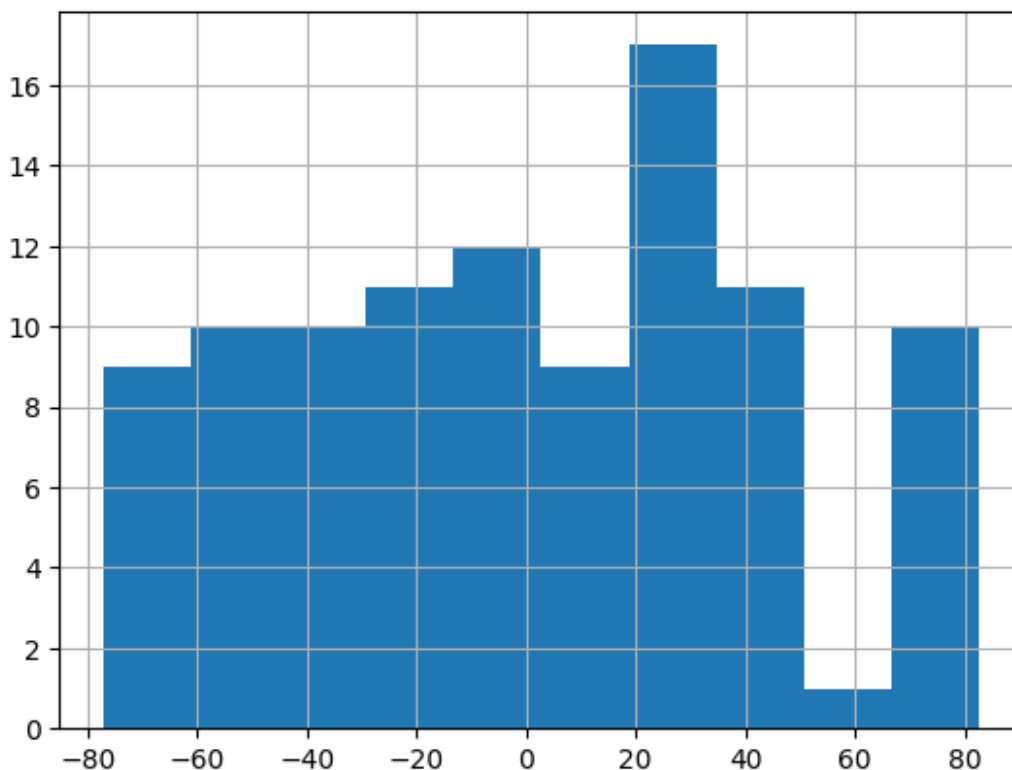
```
[ 0.32633701 -0.62809127  0.70640322]
[ 0.73826204 -0.29734633 -0.6054373 ]]
```

The principal component vectors shown are orthogonal to each other (their dot product is basically zero). Here, we check that using the first two vectors.

```
print(np.array(pca3.components_[0]).dot(np.array(pca3.components_[1])))
-1.6653345369377348e-16
```

We can use the projection onto the first principal component as if it were a new feature. For instance, we could plot a histogram of the data against the magnitude of the projection onto the first principal component. In fact, for some purposes, this magnitude (which we have called v) might make a better feature for data analysis than either height, width, or root. We must subtract off the mean before the dot product; the understanding is that this represents the component of the data relative to the mean.

```
((df2 - df2.mean()).dot(np.array(pca.components_[0]))).hist()
<Axes: >
```



```
# Showing that this is the same as the value we've called "v"
(df_example["v"] - (df2 -
df2.mean()).dot(np.array(pca.components_[0]))).max()

np.float64(0.0)
```

Linear Regression

Now let's run a linear regression model to predict the root from the height and width, as well as predicting the third principal component from the first and second. Which is easier to predict?

Predicting the root from the height and width

```
linreg = linear_model.LinearRegression()
linreg.fit(df2[['height', 'width']], df2['root'])
preds = linreg.predict(df2[['height', 'width']])
metrics.mean_squared_error(preds, df2['root']) # Mean squared error as
a metric

138.22487356329611

np.corrcoef(preds, df2['root']) # Correlation coefficient as a metric

array([[1.          , 0.79468295],
       [0.79468295, 1.          ]])
```

Predicting the third principal component from the first and second

```
first_principal_component_score = (df2 -
df2.mean()).dot(np.array(pca3.components_[0]))
second_principal_component_score = (df2 -
df2.mean()).dot(np.array(pca3.components_[1]))
third_principal_component_score = (df2 -
df2.mean()).dot(np.array(pca3.components_[2]))

linreg_first = linear_model.LinearRegression()
pc_X = np.array([first_principal_component_score,
second_principal_component_score]).T
linreg_first.fit(pc_X, third_principal_component_score)
preds = linreg_first.predict(pc_X)
metrics.mean_squared_error(preds, third_principal_component_score) #
The mean squared error is a bit smaller here.

78.74928905273173

np.corrcoef(preds, third_principal_component_score) # More
importantly, the correlation is basically zero

array([[1.00000000e+00, 1.55495941e-14],
       [1.55495941e-14, 1.00000000e+00]])
```

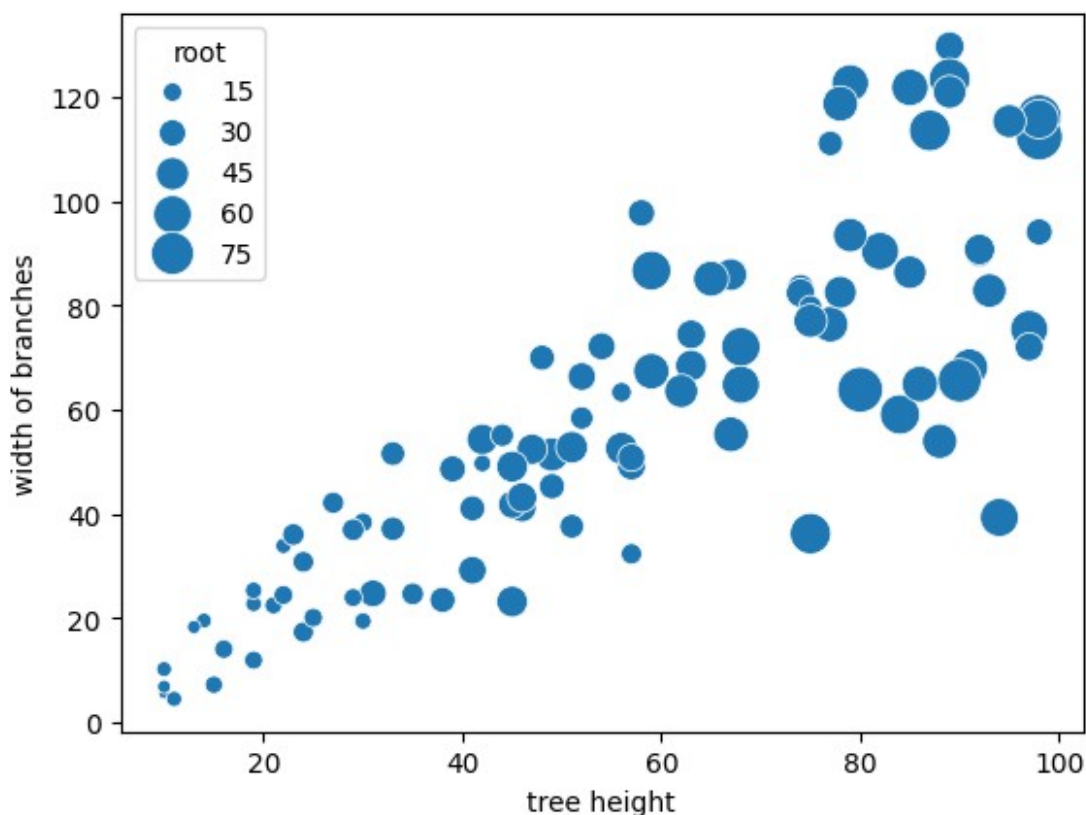

Here, we see that the projections of the first two principal components have no ability to predict the projection of the third, when measured by correlation. This is a special property of principal components. It is intended to ensure that each component gives new information not present in the previous components.

The `mean_squared_error` is misleading - it makes it look like the error is less for the second computation than the first; but the correlation coefficient is giving more useful information here. We cannot compare the mean squared error for two totally different computations, because mean squared error is highly dependent on the magnitudes of the values in question!

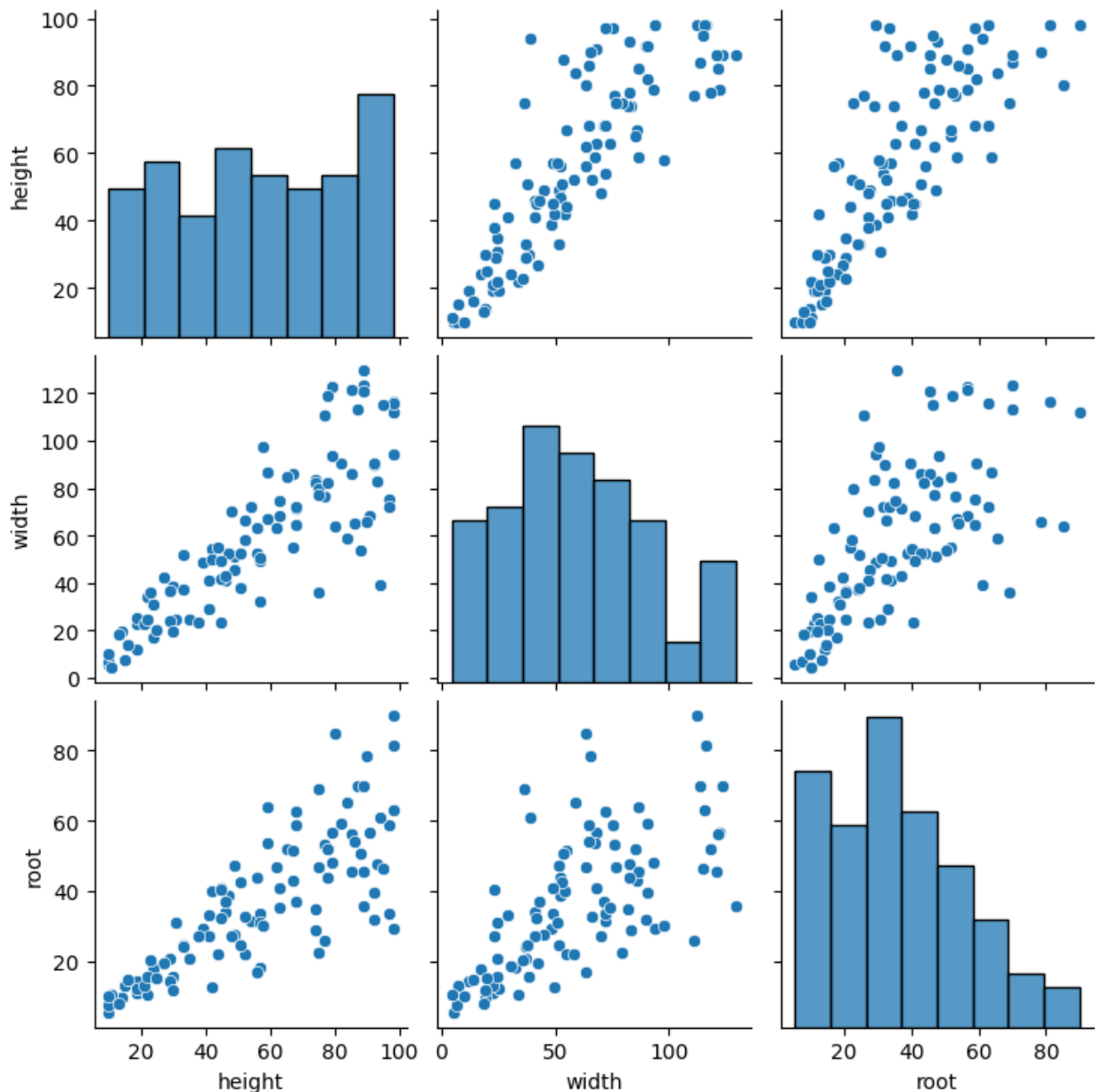
2. Weekly graph question

The tree data above can be graphed via either a pairplot or a bubble plot. What are the advantages and disadvantages of each?

```
ax = sns.scatterplot(data = df2, x = "height", y = "width", size =  
"root", sizes = (minroot / maxroot * maxbubble, maxbubble))  
ax.set_xlabel("tree height")  
ax.set_ylabel("width of branches")  
Text(0, 0.5, 'width of branches')
```



```
sns.pairplot(df2)
<seaborn.axisgrid.PairGrid at 0x2b07b8671d0>
```



The Pairplot shows the relationships between each of the variables, as well as the histograms of the variables. This captures both Univariate, and Bivariate data. The issues with the Pairplot are that the data is not well represented with accurate axes. The bubble plot shows three variables compared to one another. This is useful for multivariate analysis. The issues are that it is harder to draw conclusions between two of the variables as they are obfuscated by the area of the bubble. You can get an idea of how they are related, but there is no actual value associated with the bubble that is readable by eye.

3. Homework - Grocery Store Correlation

In a particular state, the price of milk is highly correlated with the price of cheese. Neither is correlated with the price of bread.

```
num_points = 1000
np.random.seed(0)
milk_series = np.maximum(np.random.normal(scale = 0.5, loc = 2.00,
size = num_points), 1)
cheese_series = milk_series * np.maximum(np.random.normal(scale = 0.1,
loc = 1.00, size = num_points), 0.5)
bread_series = np.maximum(np.random.normal(scale = 1.0, loc = 3.00,
size = num_points), 1)
df3 = pd.DataFrame({"milk": milk_series, "cheese": cheese_series,
"bread": bread_series})
```

Heatmap

Draw a heatmap where the x-axis category is a discretized version of the milk price and the y-axis category is a discretized version of the cheese price. That is, you can choose "low", "medium", and "high" for the milk price, or, if you prefer, you can choose 1, 2, 3, 4, and 5 for the five quintiles. (The first quintile should include the bottom fifth of the data, the second quintile contains the next fifth, and so on).

Try Googling "how do I change numerical to categorical data" to get advice on this.

```
print(milk_series.max())
print(milk_series.min())

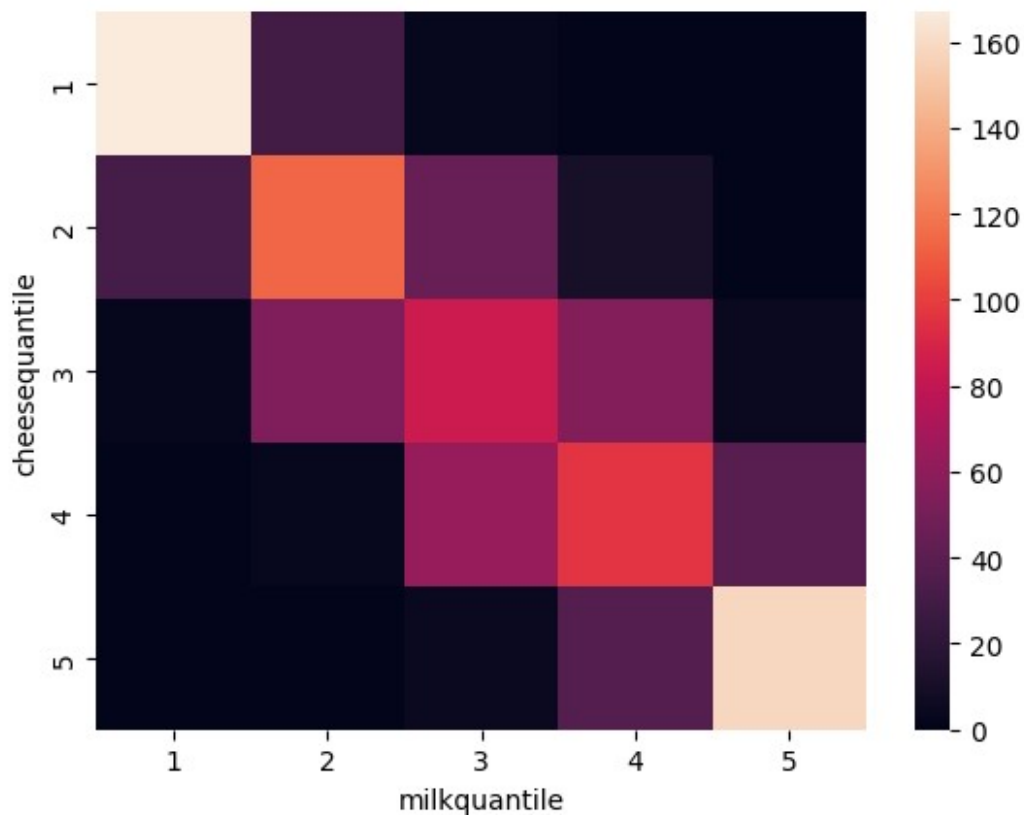
3.3796775570107913
1.0

df3['milkquantile'] = pd.qcut(df3['milk'], q=5, labels=[1,2,3,4,5])
df3['cheesequantile'] = pd.qcut(df3['cheese'], q=5,
labels=[1,2,3,4,5])
pt =
df3.groupby(['milkquantile', 'cheesequantile']).size().unstack(fill_val
ue=0)

C:\Users\thyde\AppData\Local\Temp\ipykernel_1816\3813834605.py:3:
FutureWarning: The default of observed=False is deprecated and will be
changed to True in a future version of pandas. Pass observed=False to
retain current behavior or observed=True to adopt the future default
and silence this warning.
    pt =
df3.groupby(['milkquantile', 'cheesequantile']).size().unstack(fill_val
ue=0)

sns.heatmap(pt.T)
```

```
<Axes: xlabel='milkquantile', ylabel='cheesequantile'>
```



Principal component analysis

Perform a principal component analysis of the data. What are the first, second, and third component vectors?

Challenge question: explain why the three vectors point as they do. (Could someone have guessed the order of the three vectors, even before running the analysis?)

```
pca4 = decomposition.PCA(n_components = 3)
pca4.fit(df3)
print(pca4.explained_variance_ratio_)
print(pca4.components_)
```

```
[0.78294443 0.16324412 0.04522839]
[[ 0.22299336  0.23834686 -0.01069345  0.66837654  0.66829873]
 [-0.00740863 -0.00431591  0.99983153  0.00438285  0.01562627]
 [ 0.15322251 -0.14385202  0.00828586  0.69140406 -0.6911738 ]]
```

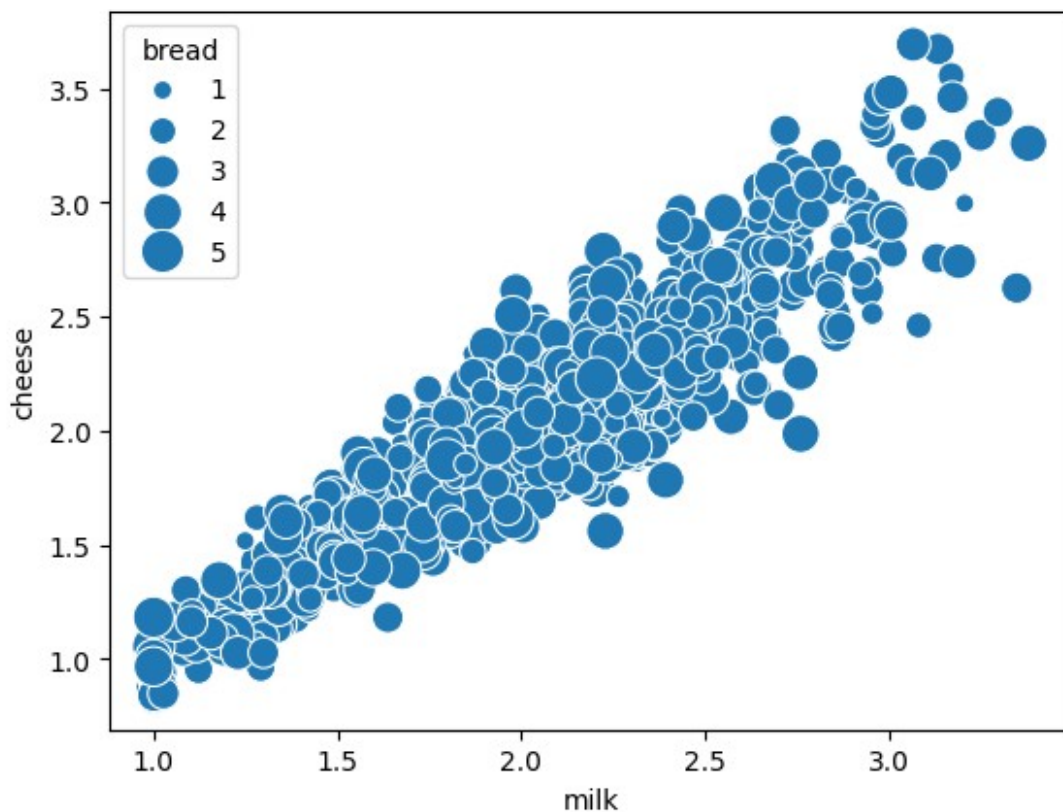
Given knowledge that milk and cheese are the largest and most correlated variables and bread is a sizeable independent variable, one could reasonably have predicted that ordering before running PCA.

Bubble plot

Try all three possible bubble plots: one where the size of the dot is determined by the milk price, one where it's the cheese price, and one where it's the bread price. Which of these plots is / are the most informative and useful?

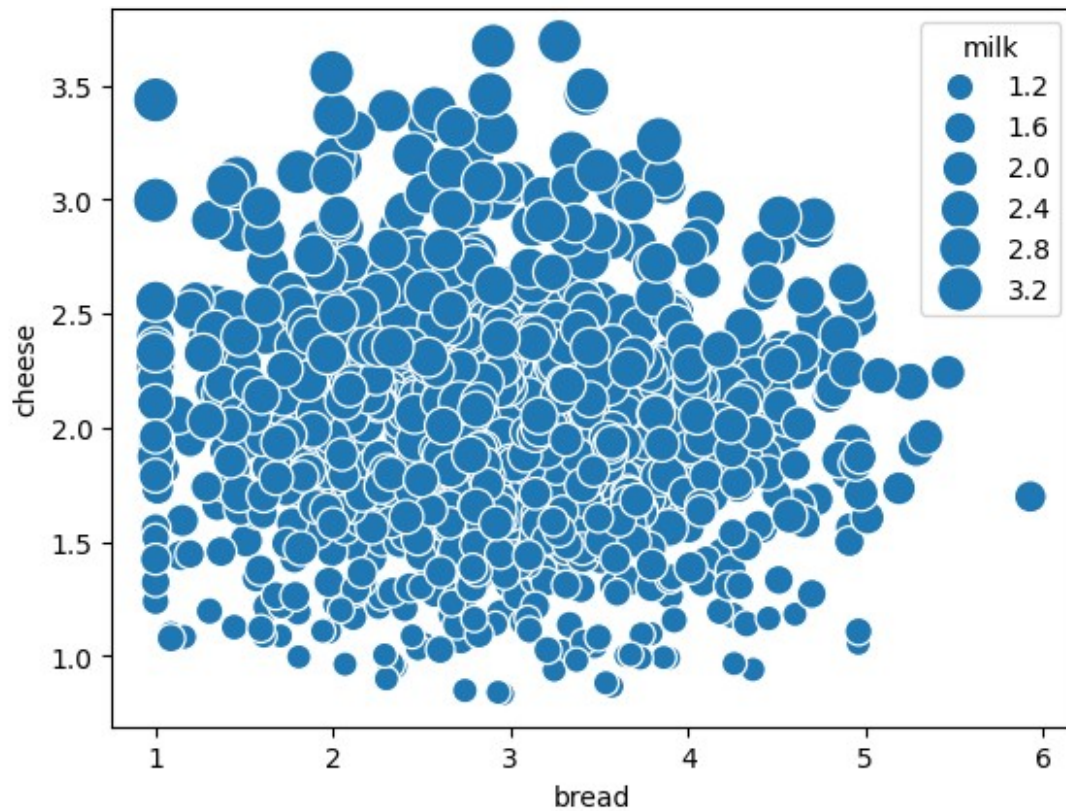
```
ax = sns.scatterplot(data = df3, x = "milk", y = "cheese", size =  
"bread", sizes = (df3['bread'].min() / df3['bread'].max() * maxbubble,  
maxbubble))  
ax.set_xlabel("milk")  
ax.set_ylabel("cheese")
```

```
Text(0, 0.5, 'cheese')
```

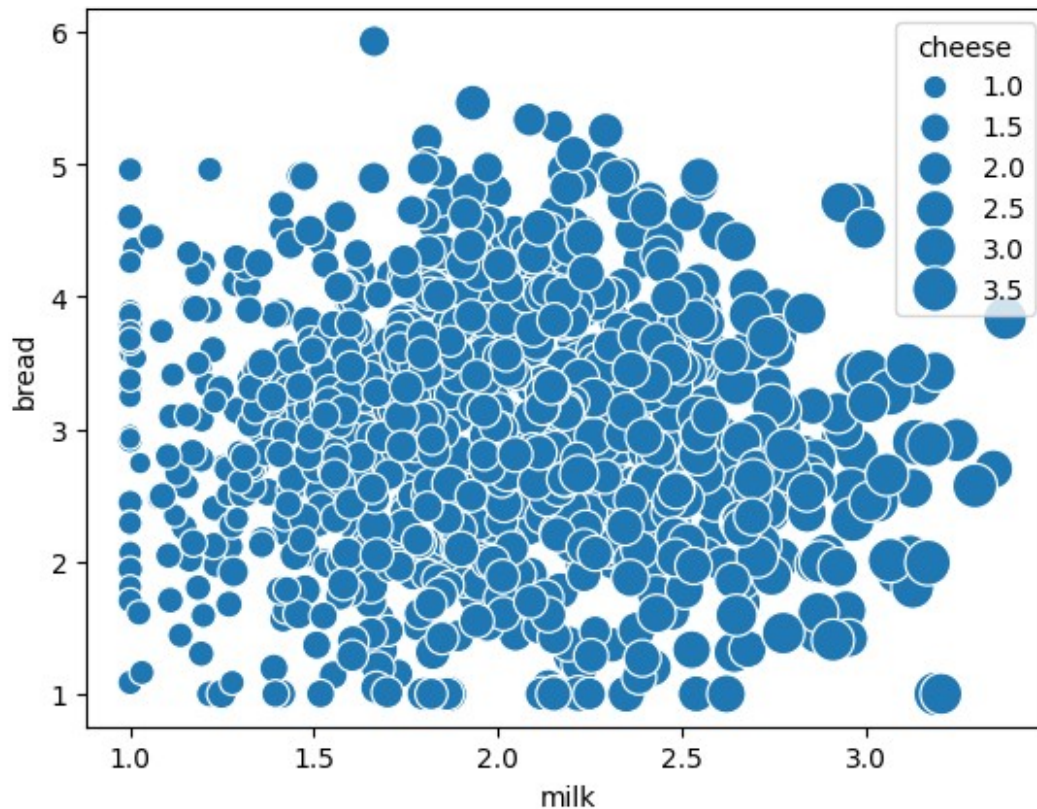


```
ax = sns.scatterplot(data = df3, x = "bread", y = "cheese", size =  
"milk", sizes = (df3['milk'].min() / df3['milk'].max() * maxbubble,  
maxbubble))  
ax.set_xlabel("bread")  
ax.set_ylabel("cheese")
```

```
Text(0, 0.5, 'cheese')
```



```
ax = sns.scatterplot(data = df3, x = "milk", y = "bread", size =  
"cheese", sizes = (df3['cheese'].min() / df3['cheese'].max() *  
maxbubble, maxbubble))  
ax.set_xlabel("milk")  
ax.set_ylabel("bread")  
Text(0, 0.5, 'bread')
```

The most informative plot is Milk x, cheese y, bread as bubble since there is a direct linear relationship between milk and cheese price.

Linear regression

Use linear regression to predict the milk price from the other two, the cheese price from the other two, and the bread price from the other two. Which prediction(s) work the best, and why?

```
linreg = linear_model.LinearRegression()
linreg.fit(df3[['milk', 'cheese']], df3['bread'])
preds1 = linreg.predict(df3[['milk', 'cheese']])
metrics.mean_squared_error(preds1, df3['bread'])
```

0.8772889513306434

```
linreg = linear_model.LinearRegression()
linreg.fit(df3[['milk', 'bread']], df3['cheese'])
preds2 = linreg.predict(df3[['milk', 'bread']])
metrics.mean_squared_error(preds2, df3['cheese'])
```

0.0396561665666383

```
linreg = linear_model.LinearRegression()
linreg.fit(df3[['bread', 'cheese']], df3['milk'])
```

```
preds3 = linreg.predict(df3[['bread', 'cheese']])
metrics.mean_squared_error(preds3, df3['milk'])

0.03447410921882385
```

The third model predicting milk price is the most effective since it has the lowest MSE. This shows that the price of cheese vs bread is accurate at predicting the milk price. Cheese and milk seem to be closely related according to the bubble plots, so this makes sense. The cheese and milk prices are largely unrelated to the bread price, which is also reflected by the high MSE of that model.

4. Storytelling With Data plot

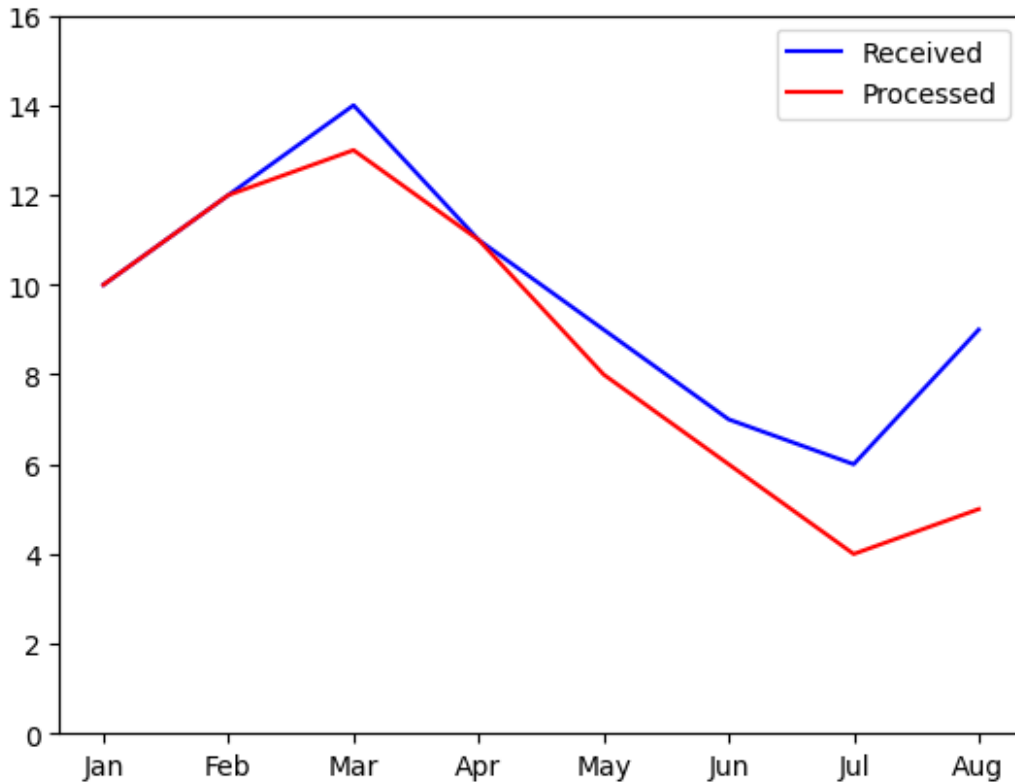
Reproduce any graph of your choice in p. 120-135 of the Storytelling With Data book as best you can. ("Use color consistently" in chapter four up to "Create a clear visual hierarchy of information" in chapter five.) You do not have to get the exact data values right, just the overall look and feel.

```
import matplotlib.pyplot as plt

received = [10, 12, 14, 11, 9, 7, 6, 9]
Processed = [10, 12, 13, 11, 8, 6, 4, 5]
Months = ["Jan", "Feb", "Mar", "Apr", "May", "Jun", "Jul", "Aug"]

plt.plot(Months, received, color="blue", label = "Received")
plt.plot(Months, Processed, color="red", label = "Processed")
plt.legend()
plt.ylim(0, 16)

(0.0, 16.0)
```

Week 9 - Multivariate Analysis, part 2

1. Lesson - no lesson this week

2. Weekly graph question

The graph below plots the first two principal component scores in a scatter plot. What can be said about the three outliers in the upper left corner of the graph? Is their first principal component score high or low? What about their second principal component score? What does that mean about their values in series_1, series_2, and series_3? It seems to me that you can say something about series_3 (what can you say?) but you may have a harder time saying something about series_1 and series_2, and an almost impossible time saying anything about the relative values of series_1 and series_2. Why is that? How are series_1 and series_2 related, according to how they were created? If you like, try drawing a pairplot for all three series and see what you get.

Overall, what are the advantages and disadvantages of the graph below? Does it show anything interesting?

I do not feel like the graph is very informative. Series 3 does not have a high correlation. The outliers are likely other outliers in the original data set. The components look similar with PC2 having additional noise. I also find it difficult to parse out which components are highly correlated within this dataset. The advantage is a 2d summary of multivariate data. The disadvantage is that you need to inspect PC3 to understand what any of this data means.

```
import numpy as np
import pandas as pd
from sklearn import decomposition
import matplotlib.pyplot as plt

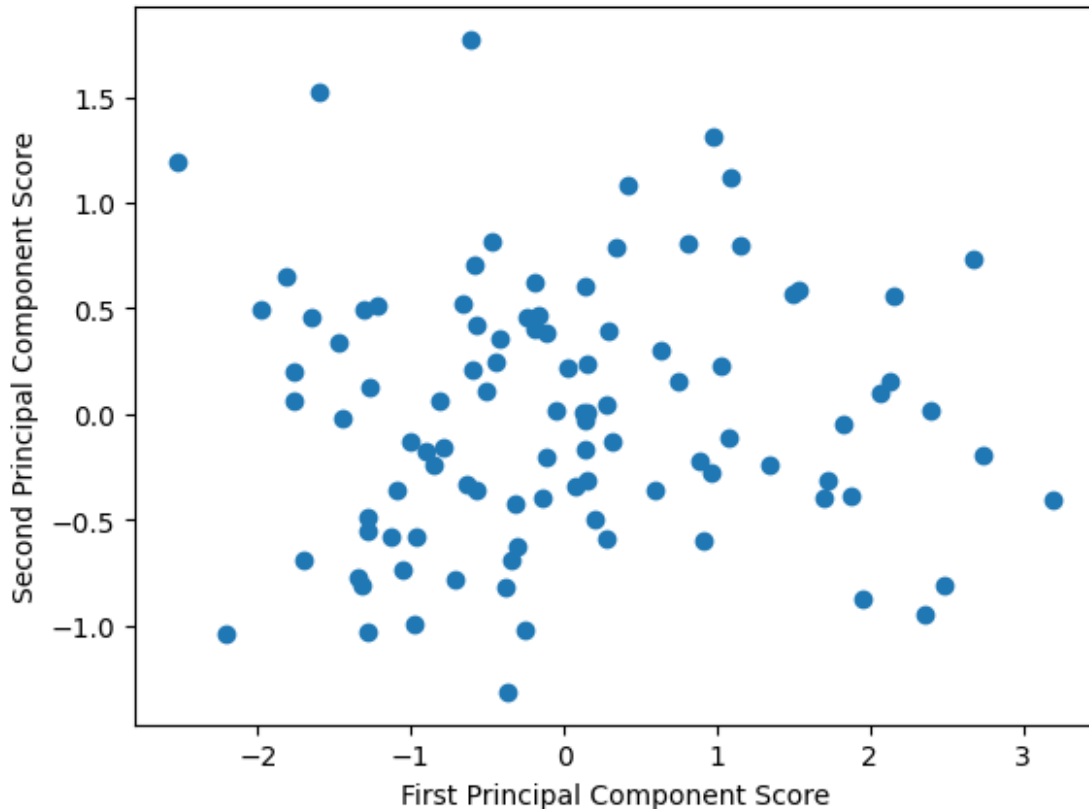
np.random.seed(0)
num_points = 100
series_1 = np.random.normal(loc = 2, scale = 0.5, size = num_points)
series_2 = series_1 * (1 + np.random.normal(loc = 0, scale = 0.1, size = num_points))
series_3 = series_1 * (1 + np.random.normal(loc = 0, scale = 0.5, size = num_points))
df = pd.DataFrame({'ser1': series_1, 'ser2': series_2, 'ser3': series_3})
df = df - df.mean() # set mean to zero, so we don't have to subtract mean from the principal component scores

pca3 = decomposition.PCA(n_components = 3)
pca3.fit(df)
print(pca3.explained_variance_ratio_)
print(pca3.components_)

first_principal_component_score = df.dot(pca3.components_[0])
second_principal_component_score = df.dot(pca3.components_[1])
plt.scatter(first_principal_component_score, second_principal_component_score)
plt.xlabel("First Principal Component Score")
plt.ylabel("Second Principal Component Score")

[0.79916477 0.18990532 0.01092991]
[[ 0.26541493  0.30096233  0.91595665]
 [ 0.60337553  0.6891417  -0.40127506]
 [ 0.75199261 -0.65917023 -0.00131519]]

Text(0, 0.5, 'Second Principal Component Score')
```



3. Working on your datasets

This week, you will do the same types of exercises as last week, but you should use your chosen datasets that someone in your class found last semester. (They likely will not be the particular datasets that you found yourself.)

Here are some types of analysis you can do: Draw heatmaps.

Draw bubble plots.

Perform Principal Component Analysis to find out the directions in which the data varies. Can you represent the data using only its projection onto its first principal component, using the methods described in Week 8? How much of the variance would this capture?

Try performing linear regression analysis using different sets of features. Which features seem most likely to be useful to predict other features?

Conclusions: Explain what conclusions you would draw from this analysis: are the data what you expect? Are the data likely to be usable? If the data are not useable, find some new data! I do not see myself continuing to use this dataset. There are very few correlations that could be used for multivariate analysis. Do you see any outliers? (Data points that are far from the rest of the data). Not really. The data is climate based so I did not see any during this part of the analysis. Does the Principal Component Analysis suggest a way to represent the data using fewer dimensions than usual - using its first one or two principal component scores, perhaps? Not that I can tell. I think

that the low correlation of the data is impacting this. Try using your correlation information from previous weeks to help choose features for linear regression.

```
df = pd.read_csv(r"C:\Users\thyde\OneDrive\Documents\OMDS Resources\DX  
699 AI for Leaders\archive\climate_change_dataset.csv")
```

```
numeric_df = df.select_dtypes(include='number')
```

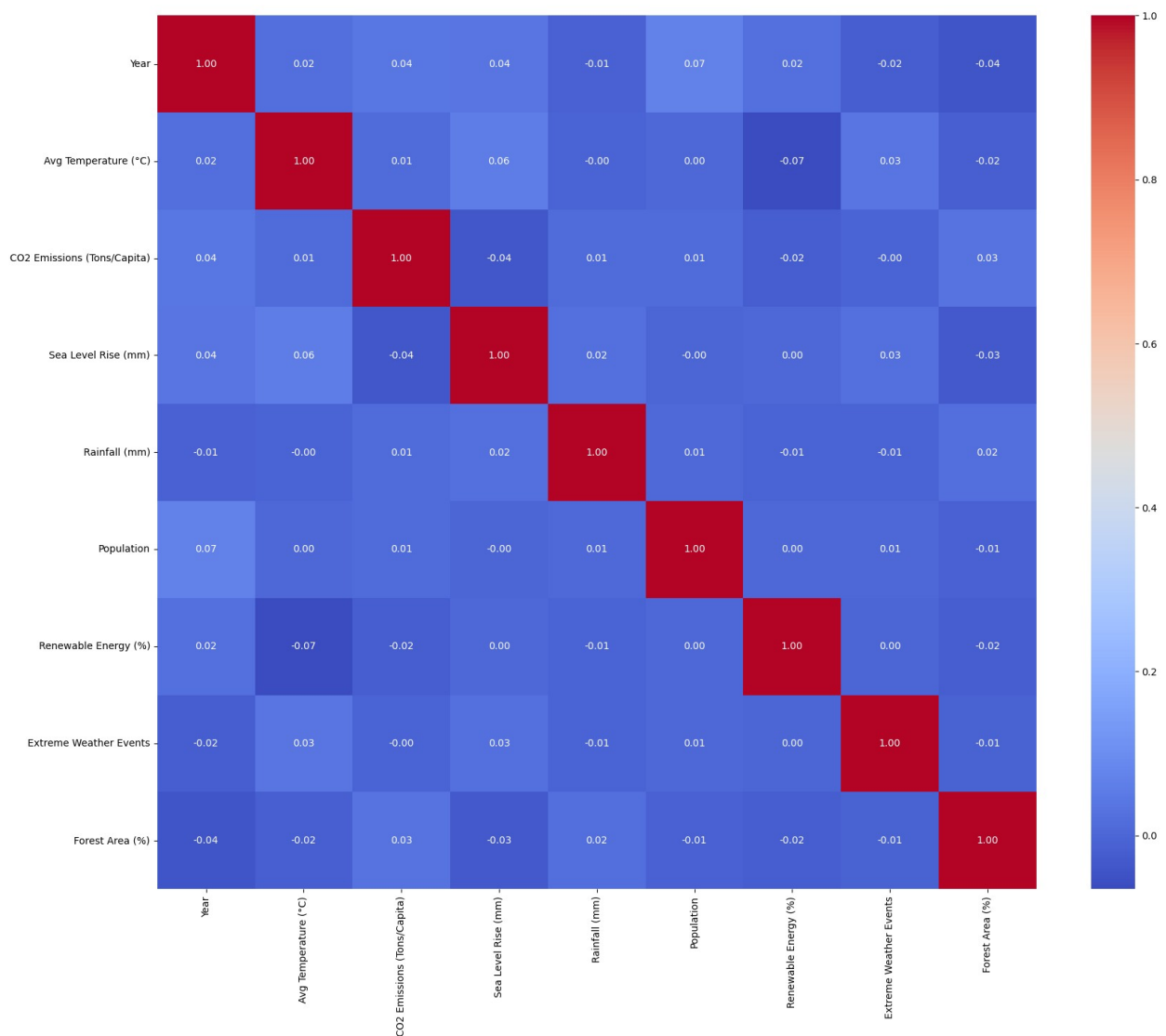
```
import seaborn as sns
```

```
#heatmap all countries
```

```
plt.figure(figsize=(20, 16))
```

```
sns.heatmap(numeric_df.corr(), annot=True, fmt="0.2f",  
cmap="coolwarm")
```

```
<Axes: >
```



Hm, I am not seeing any strong correlation. Maybe I should consider nixing this dataset for Milestone 4. I am going to filter to USA only next.

```
numeric_df.head()
```

	Year	Avg Temperature (°C)	C02 Emissions (Tons/Capita)	\
0	2006	8.9	9.3	
1	2019	31.0	4.8	
2	2014	33.9	2.8	
3	2010	5.9	1.8	
4	2007	26.9	5.6	

	Sea Level Rise (mm)	Rainfall (mm)	Population	Renewable Energy (%)	\
0	3.1	1441	530911230	20.4	
1	4.2	2407	107364344	49.2	
2	2.2	1241	441101758	33.3	
3	3.2	1892	1069669579	23.7	
4	2.4	1743	124079175	12.5	

	Extreme Weather Events	Forest Area (%)
0	14	59.8
1	8	31.0
2	9	35.5
3	7	17.7
4	4	17.4

```
df.head()
```

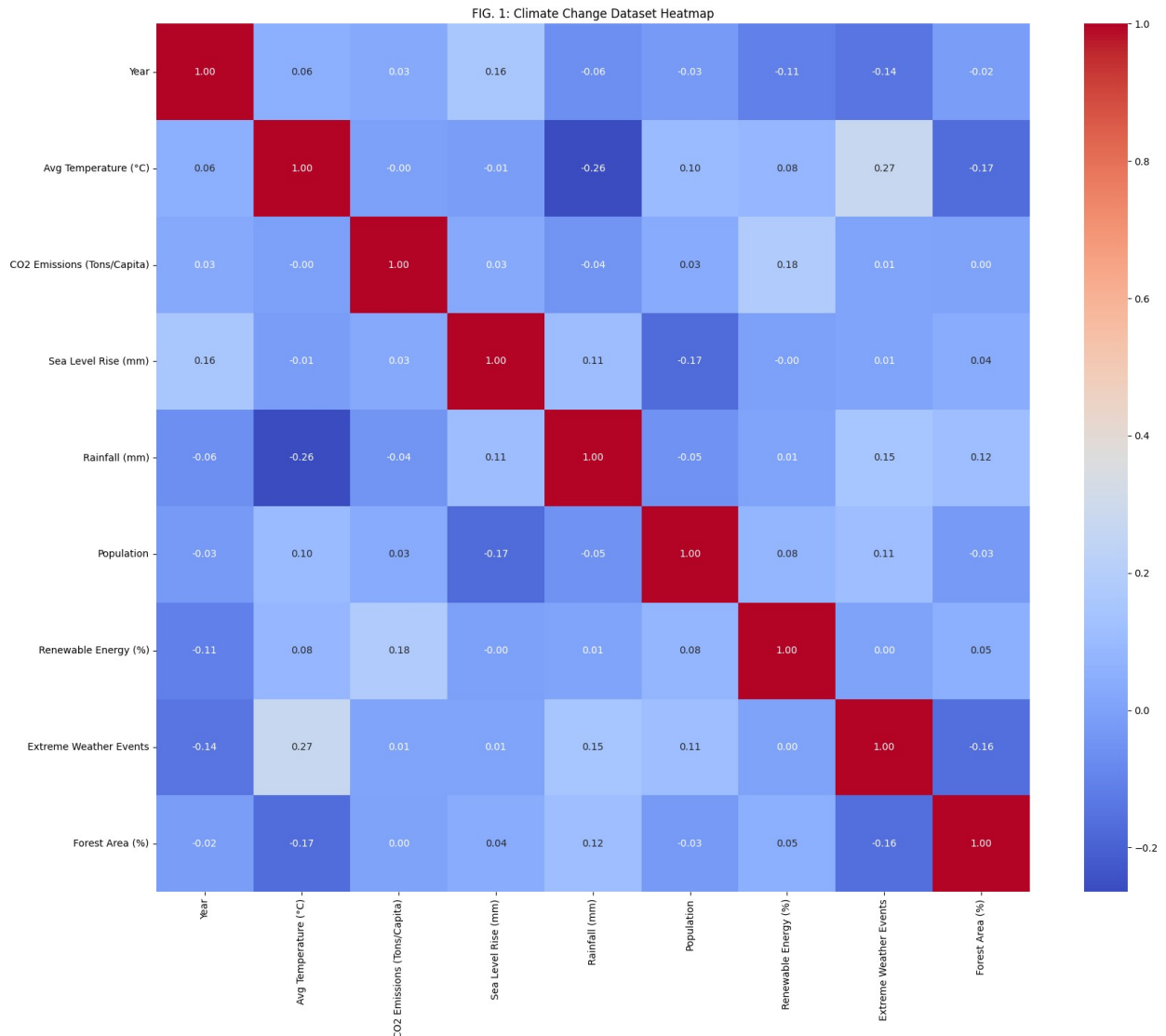
	Year	Country	Avg Temperature (°C)	C02 Emissions (Tons/Capita)
0	2006	UK	8.9	9.3
1	2019	USA	31.0	4.8
2	2014	France	33.9	2.8
3	2010	Argentina	5.9	1.8
4	2007	Germany	26.9	5.6

	Sea Level Rise (mm)	Rainfall (mm)	Population	Renewable Energy (%)	\
0	3.1	1441	530911230	20.4	

1	4.2	2407	107364344
49.2			
2	2.2	1241	441101758
33.3			
3	3.2	1892	1069669579
23.7			
4	2.4	1743	124079175
12.5			

	Extreme Weather Events	Forest Area (%)
0	14	59.8
1	8	31.0
2	9	35.5
3	7	17.7
4	4	17.4

```
df_usa = df[df["Country"] == "USA"]
df_usa_numeric = df_usa.select_dtypes(include='number')
plt.figure(figsize=(20, 16))
sns.heatmap(df_usa_numeric.corr(), annot=True, fmt="0.2f",
cmap="coolwarm")
plt.title("FIG. 1: Climate Change Dataset Heatmap")
Text(0.5, 1.0, 'FIG. 1: Climate Change Dataset Heatmap')
```



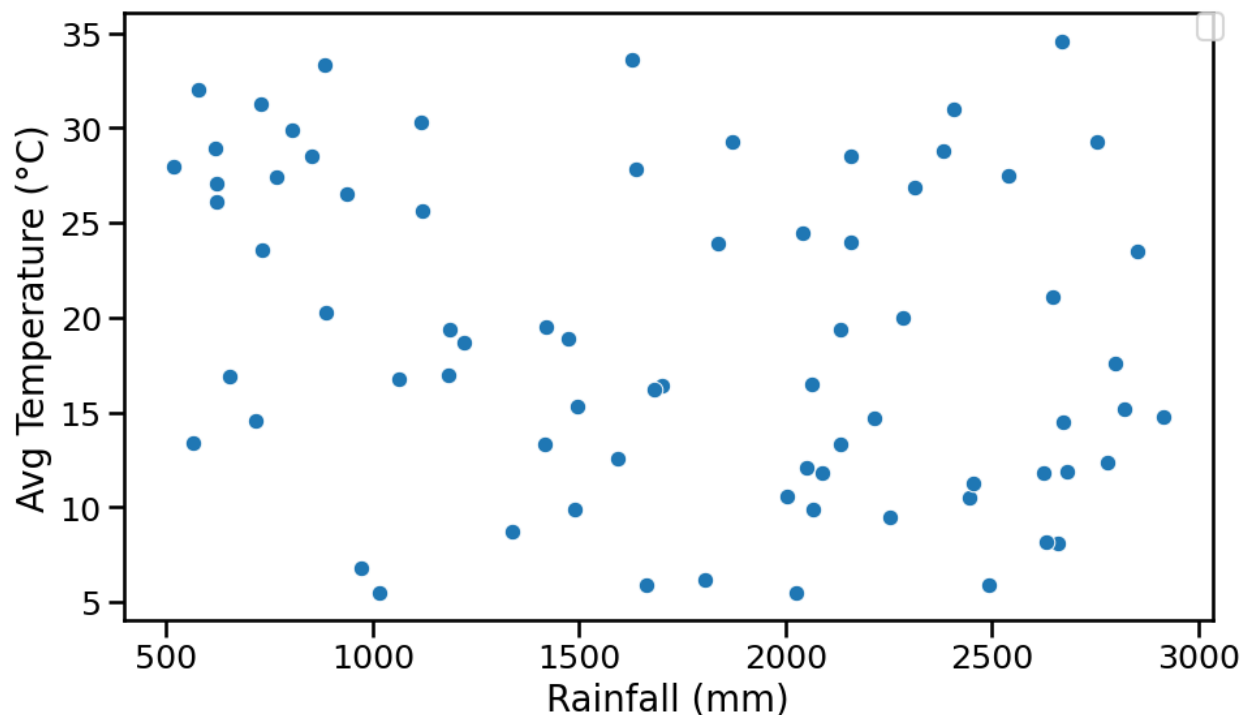
#this is a bubble plot. I am unsure that this data will be good to proceed with.

```
sns.set_context("talk", font_scale=1.1)
plt.figure(figsize=(10, 6))
sns.scatterplot(x="Rainfall (mm)",
                y="Avg Temperature (°C)",
                data=df_usa_numeric)
# Put the legend out of the figure
plt.legend(bbox_to_anchor=(1.01, 1), borderaxespad=0)
plt.xlabel("Rainfall (mm)")
plt.ylabel("Avg Temperature (°C)")
plt.tight_layout()
```

C:\Users\thyde\AppData\Local\Temp\ipykernel_31884\4050534510.py:8:
 UserWarning: No artists with labels found to put in legend. Note that
 artists whose label start with an underscore are ignored when legend()

is called with no argument.

```
plt.legend(bbox_to_anchor=(1.01, 1), borderaxespad=0)
```



Below I perform PCA

```
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler

X=df_usa_numeric.values
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

print(f"Original shape: {X_scaled.shape}")
print(f"Reduced shape: {X_pca.shape}")
print(f"Explained variance ratio: {pca.explained_variance_ratio_}")

Original shape: (73, 9)
Reduced shape: (73, 2)
Explained variance ratio: [0.17112642 0.14207745]
```

Finally I will try to get a linear regression out of this data. I will likely fail due to the data's correlation problems but we will try.

```
from sklearn.linear_model import LinearRegression
```



```

from sklearn.metrics import mean_squared_error
from sklearn.model_selection import train_test_split

xlinreg = df_usa_numeric.drop('Avg Temperature (°C)', axis=1)
ylinreg = df_usa_numeric[['Avg Temperature (°C)']]

linregmodel = LinearRegression()
linregmodel.fit(xlinreg,ylinreg)
LinearRegression()

```

I chose avg temp as the target as that is the focus of my research question.

```

df_agr = pd.read_csv(r"C:\Users\thyde\OneDrive\Documents\OMDS
Resources\DX 699 AI for Leaders\archive\
climate_change_impact_on_agriculture_2024 -
climate_change_impact_on_agriculture_2024.csv")

numeric_df_agr = df_agr.select_dtypes(include="number")
numeric_df_agr.head()

```

	Year	Average Temperature (C)	Total Precipitation (mm)	\
0	1997	23.46	1816.41	
1	1996	21.59	666.95	
2	1999	18.62	576.51	
3	1998	0.15	1838.65	
4	2019	2.33	839.77	

	C02 Emissions (MT)	Crop Yield (MT per HA)	Extreme Weather Events	\
0	27.70	1.611		2
1	28.44	1.584		10
2	7.29	4.530		3
3	17.36	2.232		8
4	25.67	2.016		5

	Irrigation Access %	Pesticide Use (KG per HA)	Fertilizer Use (KG per HA)	\
0	20.22	0.56		
1	41.15	18.30		
2	33.33	1.59		

3	11.62	9.55
80.77		
4	18.40	8.92
70.37		

	Soil Health Index	Economic Impact (Million USD)
0	79.09	480.61
1	35.89	384.91
2	65.14	1100.96
3	89.48	687.85
4	82.21	524.89

```
df_usa_agr = df_agr[df_agr["Country"] == "USA"]
```

```
df_usa_agr_numeric = df_usa_agr.select_dtypes(include="number")
```

```
df_usa_agr_numeric.head()
```

	Year	Average Temperature (C)	Total Precipitation (mm)	\
8968	2019	17.19	975.13	
8969	2003	14.70	2272.28	
8970	1990	12.03	2704.89	
8971	2024	12.18	1647.58	
8972	2023	30.62	1630.22	

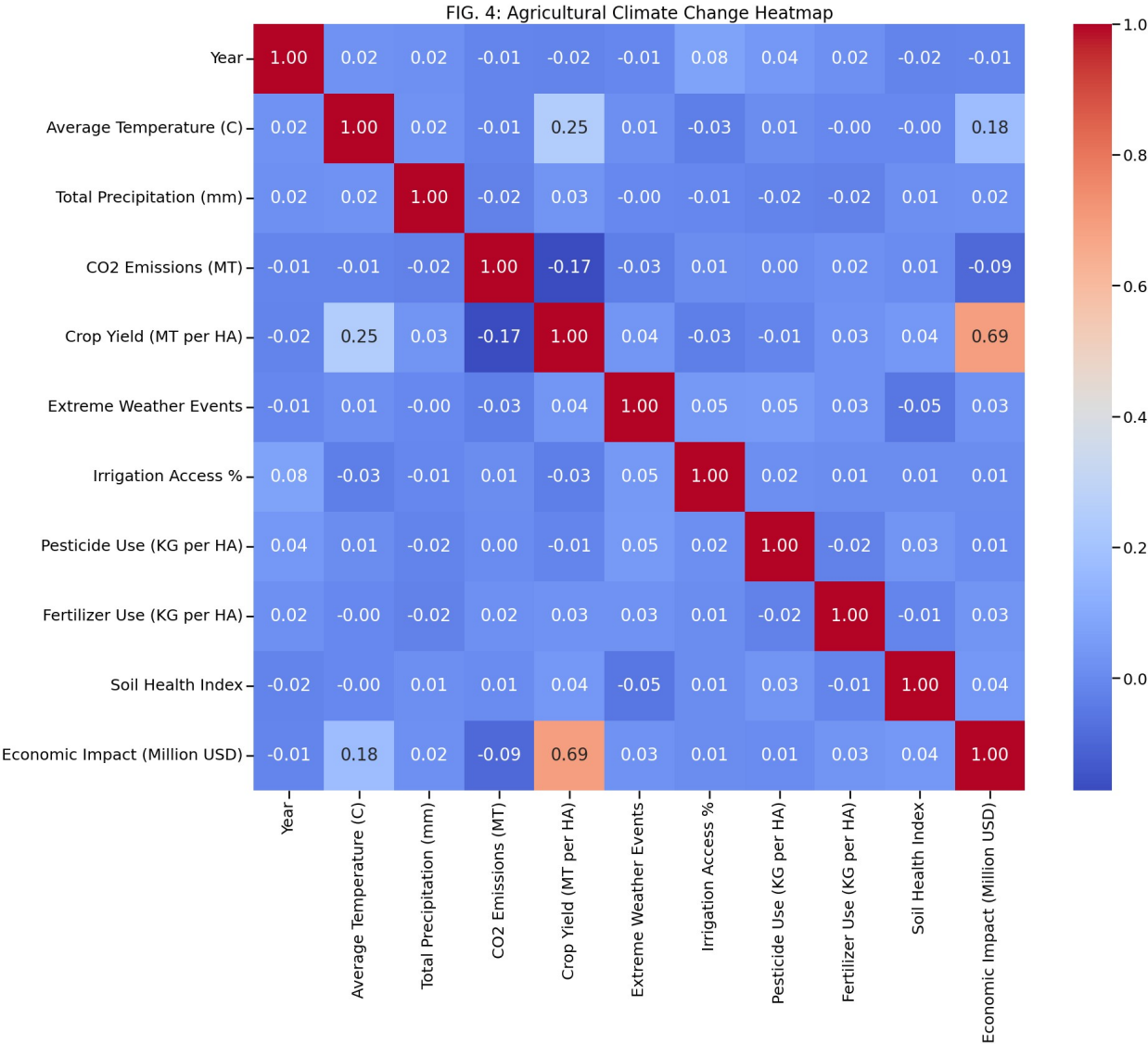
	C02 Emissions (MT)	Crop Yield (MT per HA)	Extreme Weather Events	\
8968	10.73	2.180		
5				
8969	11.45	2.500		
9				
8970	3.86	2.450		
8				
8971	19.10	1.872		
8				
8972	29.28	3.249		
9				

	Irrigation Access %	Pesticide Use (KG per HA)	\
8968	52.45	26.06	
8969	57.37	19.48	
8970	54.67	13.10	
8971	36.61	43.15	
8972	70.31	5.80	

	Fertilizer Use (KG per HA)	Soil Health Index	\
8968	71.56	97.32	
8969	46.13	77.32	
8970	2.56	50.75	
8971	22.46	52.18	
8972	49.85	31.92	

Economic Impact (Million USD)	
8968	353.16
8969	1126.27
8970	1026.35
8971	925.79
8972	509.77

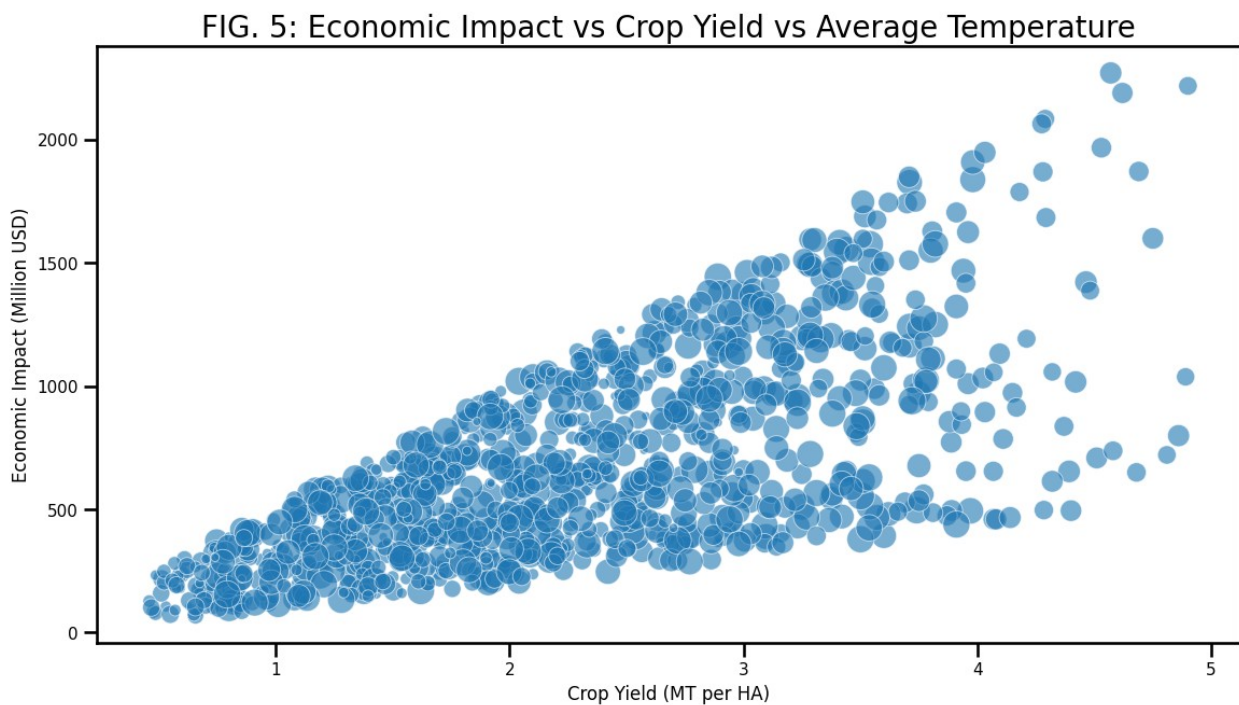
```
plt.figure(figsize=(20, 16))
sns.heatmap(df_usa_agr_numeric.corr(), annot=True, fmt="0.2f",
cmap="coolwarm")
plt.title("FIG. 4: Agricultural Climate Change Heatmap")
Text(0.5, 1.0, 'FIG. 4: Agricultural Climate Change Heatmap')
```



```

import matplotlib.pyplot as plt
plt.figure(figsize=(12,7))
temps = df_usa_agr_numeric['Average Temperature (C)']
norm = (temps - temps.min()) / (temps.max() - temps.min())
marker_sizes = norm * (350 - 30) + 30
ax = sns.scatterplot(data=df_usa_agr_numeric,
x="Crop Yield (MT per HA)",
y="Economic Impact (Million USD)",
size=marker_sizes,
sizes=(marker_sizes.min(), marker_sizes.max()),
alpha=0.6,
legend=False)
ax.set_title("FIG. 5: Economic Impact vs Crop Yield vs Average Temperature")
ax.set_xlabel("Crop Yield (MT per HA)", fontsize=12)
ax.set_ylabel("Economic Impact (Million USD)", fontsize=12)
ax.tick_params(labelsize=11)
ax.margins(0.05)
plt.tight_layout()
plt.show()

```



```

pca2 = decomposition.PCA(n_components = 3)
pca2.fit(df_usa_agr_numeric)
print(pca2.explained_variance_ratio_)
print(pca2.components_)

```

```

[0.79388076 0.20308087 0.00102104]
[[ 2.44294248e-04  3.81863067e-04  9.99921178e-01 -2.79799250e-04
  4.47143618e-05 -1.53615085e-05 -3.53140022e-04 -3.43896610e-04
 -7.97915943e-04  3.46726708e-04  1.25041047e-02]
 [-3.53423211e-04  5.05667961e-03 -1.25050459e-02 -2.00348410e-03
  1.66191282e-03  2.25158507e-04  5.51434502e-04  2.45261688e-04
  2.18977768e-03  2.15804771e-03  9.99900638e-01]
 [ 9.26231800e-03 -4.60757341e-03  8.42076374e-04  6.81426486e-03
  2.55653648e-04  3.06898745e-03  4.69403560e-02 -1.55033442e-02
  9.98540988e-01 -1.74354930e-02 -2.12160997e-03]]

df_C02 = pd.read_csv(r"C:\Users\thyde\OneDrive\Documents\OMDS
Resources\DX 699 AI for Leaders\archive\owid-co2-data.csv")

numeric_df_C02 = df_C02.select_dtypes(include='number')

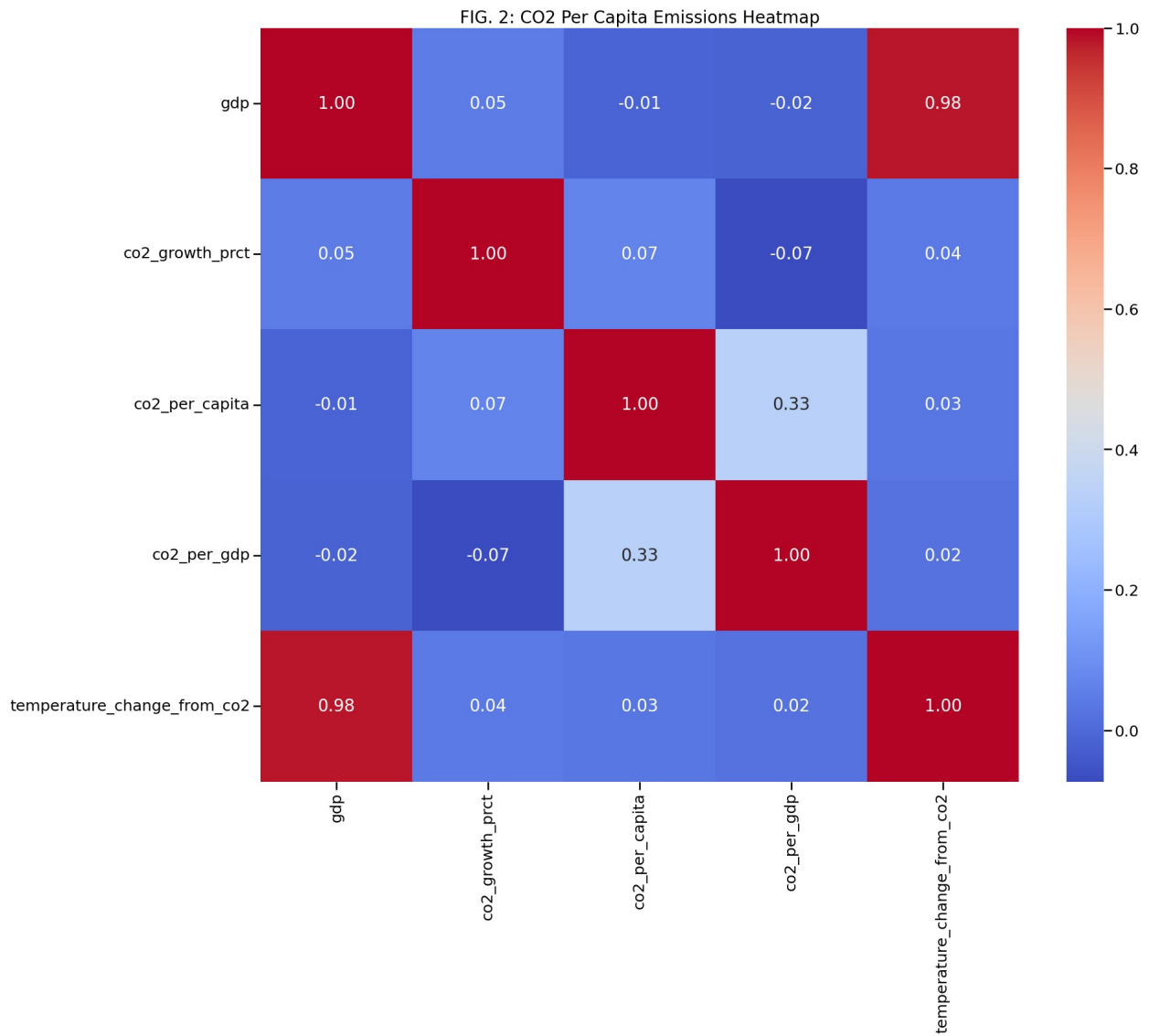
nona_numeric_df_C02 = numeric_df_C02.dropna()

clean_numeric_dfC02 =
nona_numeric_df_C02[['gdp', 'co2_growth_prct', 'co2_per_capita', 'co2_per
_gdp', 'temperature_change_from_co2']].copy()

plt.figure(figsize=(20, 16))
sns.heatmap(clean_numeric_dfC02.corr(), annot=True, fmt="0.2f",
cmap="coolwarm")
plt.title("FIG. 2: C02 Per Capita Emissions Heatmap")

Text(0.5, 1.0, 'FIG. 2: C02 Per Capita Emissions Heatmap')

```



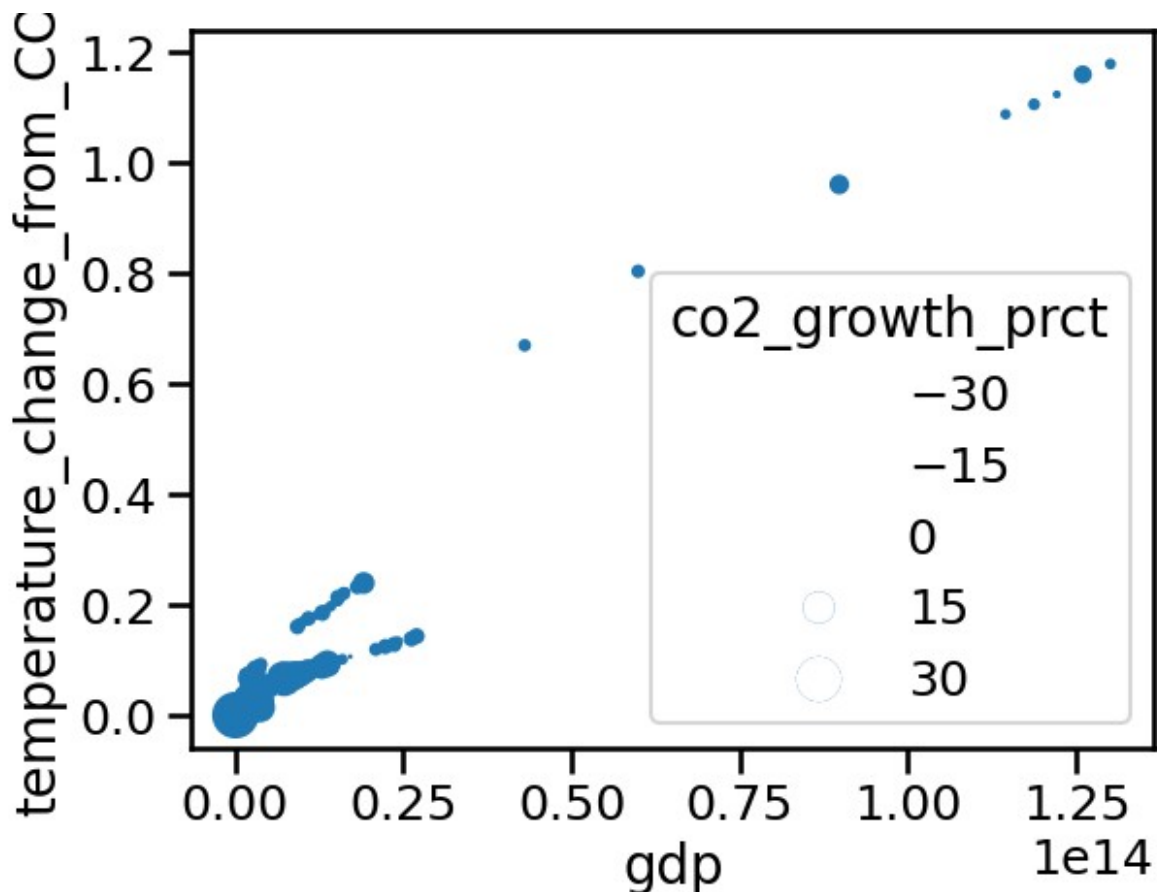
```
ax = sns.scatterplot(data = clean_numeric_dfCO2, x = "gdp", y =
"temperature_change_from_co2", size = "co2_growth_prct", sizes =
(clean_numeric_dfCO2['co2_growth_prct'].min() /
clean_numeric_dfCO2['co2_growth_prct'].max() * 300, 300))
ax.set_xlabel("gdp")
ax.set_ylabel("temperature_change_from_CO2")
```

```
C:\Users\thyde\AppData\Local\Packages\
PythonSoftwareFoundation.Python.3.11_qbz5n2kfra8p0\LocalCache\local-
packages\Python311\site-packages\matplotlib\collections.py:999:
RuntimeWarning: invalid value encountered in sqrt
    scale = np.sqrt(self._sizes) * dpi / 72.0 * self._factor
C:\Users\thyde\AppData\Local\Packages\
PythonSoftwareFoundation.Python.3.11_qbz5n2kfra8p0\LocalCache\local-
packages\Python311\site-packages\seaborn\relational.py:456:
RuntimeWarning: invalid value encountered in sqrt
```

```

linewidth = .08 * np.sqrt(np.percentile(sizes, 10))
C:\Users\thyde\AppData\Local\Packages\
PythonSoftwareFoundation.Python.3.11_qbz5n2kfra8p0\LocalCache\local-
packages\Python311\site-packages\seaborn\utils.py:868: RuntimeWarning:
invalid value encountered in sqrt
    "markersize": np.sqrt(kws.pop("s", rc["lines.markersize"] ** 2)),
Text(0, 0.5, 'temperature_change_from_CO2')

```



```

import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

plt.figure(figsize=(12, 7))

sizes_series = clean_numeric_dfCO2['co2_per_gdp'].fillna(0)
if sizes_series.max() == sizes_series.min():
    # all equal - use a constant reasonable size
    marker_sizes = np.full_like(sizes_series, 120, dtype=float)
else:

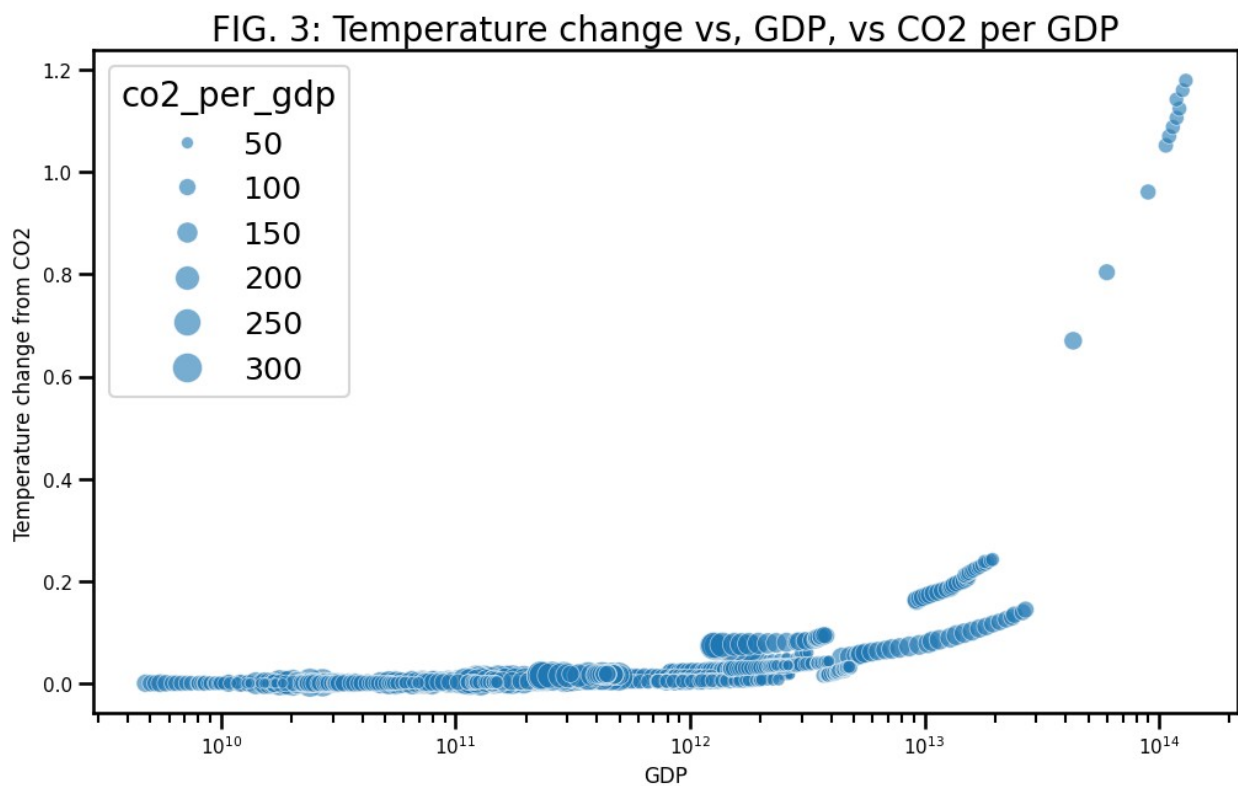
```

```

norm = (sizes_series - sizes_series.min()) / (sizes_series.max() -
sizes_series.min())
min_size, max_size = 30, 300 # adjust these for smaller/larger
bubbles
marker_sizes = norm * (max_size - min_size) + min_size

ax = sns.scatterplot(
data=clean_numeric_dfCO2,
x="gdp",
y="temperature_change_from_co2",
size=marker_sizes, # pass the precomputed sizes
sizes=(marker_sizes.min(), marker_sizes.max()),
alpha=0.6, # transparency to reduce overplotting
legend=True # hide default size legend (create custom
if needed)
)
ax.set_title("FIG. 3: Temperature change vs, GDP, vs CO2 per GDP")
ax.set_xscale('log')
ax.set_xlabel("GDP", fontsize=12)
ax.set_ylabel("Temperature change from CO2", fontsize=12)
ax.tick_params(labelsize=11)

```




```
df_emissions = pd.read_csv(r"C:\Users\thyde\OneDrive\Documents\OMDS
Resources\DX 699 AI for Leaders\archive\
emissions_low_granularity.csv")
```

```
df_emissions.head()
```

	year	parent_entity	parent_type	\
0	1962	Abu Dhabi National Oil Company	State-owned Entity	
1	1963	Abu Dhabi National Oil Company	State-owned Entity	
2	1964	Abu Dhabi National Oil Company	State-owned Entity	
3	1965	Abu Dhabi National Oil Company	State-owned Entity	
4	1966	Abu Dhabi National Oil Company	State-owned Entity	

	lei	total_emissions_MtCO2e
0	549300901JMLWDLTIC26	0.498240
1	549300901JMLWDLTIC26	1.050222
2	549300901JMLWDLTIC26	4.174018
3	549300901JMLWDLTIC26	6.193849
4	549300901JMLWDLTIC26	7.562050

4. Storytelling With Data plot

Reproduce any graph of your choice in p. 136-150 of the Storytelling With Data book as best you can. ("The power of super-categories" to the end of chapter 5). You do not have to get the exact data values right, just the overall look and feel.

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.patches import FancyBboxPatch

# Sample data (change to your values)
categories = ["Category 1", "Category 2", "Category 3", "Category 4",
"Category 5"]
values = [7, 5, 4, 4, 3]

# Style settings
track_color = "#bdbdbd"           # gray track
brand_color = "#7fbf7f"          # green brand bar
value_text_color = "white"
label_color = "#6d6d6d"
title_color_grey = "#4a4a4a"
title_color_brand = brand_color
footer_color = title_color_brand

fig, ax = plt.subplots(figsize=(5, 3)) # adjust size as needed
y_pos = np.arange(len(categories))[::-1] # reverse so first category
on top
```

```

# Parameters for bar geometry
track_width = 10 # full scale for the track background
bar_height = 0.6
corner_rounding = 0.15 # box roundness for FancyBboxPatch

# Draw tracks (full-length rounded rectangles)
for i, y in enumerate(y_pos):
    rect = FancyBboxPatch(
        (0, y - bar_height/2),
        track_width,
        bar_height,

boxstyle=f"round,pad=0,rounding_size={bar_height*corner_rounding}",
        linewidth=0,
        facecolor=track_color,
        zorder=0
    )
    ax.add_patch(rect)

# Draw brand-colored filled bars proportional to value (scale values
to track_width)
max_val = max(values)
scale = track_width / max_val
for i, (v, y) in enumerate(zip(values[::-1], y_pos)):
    filled_w = v * scale
    rect = FancyBboxPatch(
        (0, y - bar_height/2),
        filled_w,
        bar_height,

boxstyle=f"round,pad=0,rounding_size={bar_height*corner_rounding}",
        linewidth=0,
        facecolor=brand_color,
        zorder=1
    )
    ax.add_patch(rect)
    # numeric label on the right inside the filled bar (if space) or
just outside
    txt_x = filled_w - 0.5 if filled_w > 1.2 else filled_w + 0.2
    ha = "right" if filled_w > 1.5 else "left"
    txt_color = value_text_color if filled_w > 1.5 else label_color
    ax.text(txt_x, y, str(v), va="center", ha=ha, color=txt_color,
    fontsize=9, weight="semibold")

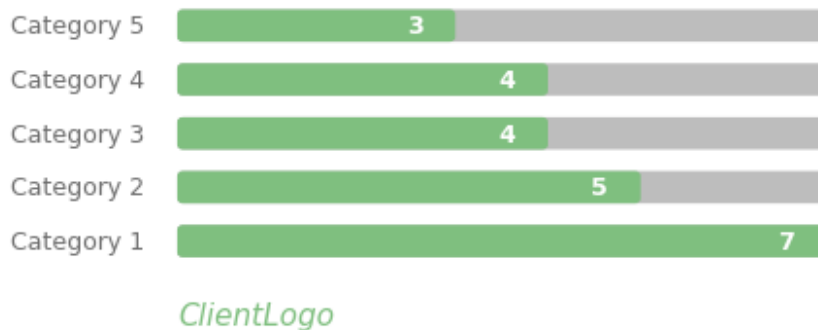
# Category labels to the left (aligned with bars)
for i, (cat, y) in enumerate(zip(categories[::-1], y_pos)):
    ax.text(-0.5, y, cat, va="center", ha="right", color=label_color,
    fontsize=9)

```

```
# Footer (ClientLogo placeholder text)
ax.text(0, -1.1, "ClientLogo", va="top", ha="left",
color=footer_color, fontsize=11, style="italic")

# Axes cleaning
ax.set_xlim(-2, track_width + 0.5)
ax.set_ylim(-1.5, len(categories) + 1)
ax.axis("off")
plt.title("Leverage Brand Color")
plt.tight_layout()
plt.show()
```

Leverage Brand Color



Week 10 - Introduction to Modeling

1. Lesson

Last week, we reviewed one model: linear regression. This week, we are going to review another, called random forest. The random forest model builds many decision trees, then averages their estimates to make a prediction.

You will also perform:

- Cross-validation
- Hyperparameter tuning
- A performance metric (root mean squared error)

How does random forest work?

A random forest is a set of decision trees. Let's say we use `n_estimators = 100` decision trees, and we have ten features. Then each decision tree is built so that a given branch gets about three ($\sqrt{10}$) features to choose from. It starts from the top of the tree, selects the best feature, then proceeds to the next nodes. The best feature is the one that minimizes the mean squared error, which must be computed for both branches, weighted according to the probability of each branch, and then summed over both branches.

```
import pandas as pd
import numpy as np
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import root_mean_squared_error
from sklearn.model_selection import GridSearchCV
from sklearn.metrics import make_scorer
```

Here, we build three series of random numbers between 0 and 1. We use a somewhat arbitrary formula to construct the synthetic data. The fourth value (the target) equals the first series minus twice the second, with the whole then being multiplied by the third series. If we stopped here, the target would be completely dependent on the three feature series - it would be easy to predict. However, we then add a random normal distribution to make the prediction harder.

There are a train series and a test series, with different features ("a" and "b"), and with different random normal data added. This tests whether our model can predict new data. We fit to the "a" features using the `train_target`, then predict the `test_target` using the "b" features.

```
np.random.seed(0)
num_points = 10000
feature_1a = np.random.random(size = num_points) * 3
feature_2a = np.random.random(size = num_points) * 3
feature_3a = np.random.random(size = num_points) * 3
train_target = (feature_1a - 2 * feature_2a) * feature_3a +
np.random.normal(size = num_points)
feature_1b = np.random.random(size = num_points) * 3
feature_2b = np.random.random(size = num_points) * 3
feature_3b = np.random.random(size = num_points) * 3
test_target = (feature_1b - 2 * feature_2b) * feature_3b +
np.random.normal(size = num_points)
train_df = pd.DataFrame({"f1": feature_1a, "f2": feature_2a, "f3":
feature_3a})
test_df = pd.DataFrame({"f1": feature_1b, "f2": feature_2b, "f3":
feature_3b})
rf = RandomForestRegressor()
rf.fit(train_df.values, train_target)

RandomForestRegressor()
```

As expected, the RMSE is smallest when predicting the training data, larger when predicting the testing data, and largest when comparing random (shuffled) data instead of using a prediction.

In the last case, there is no attempt to make a good prediction. The small RMSE when predicting the training data indicates that the model has "memorized" the training data to some degree.

```
print("RMSE when predicting training data:")
print(root_mean_squared_error(rf.predict(train_df.values),
train_target))
print("RMSE when predicting testing data:")
print(root_mean_squared_error(rf.predict(test_df.values),
test_target))
test_target_shuffle = test_target.copy()
np.random.shuffle(test_target_shuffle)
print("RMSE with random predictions:")
print(root_mean_squared_error(test_target_shuffle, test_target))
```

```
RMSE when predicting training data:
0.4044649864687342
RMSE when predicting testing data:
1.0666272939687746
RMSE with random predictions:
5.262103337484077
```

Next, we perform a Grid Search over various parameters. We've used a small number of parameters here, but you could add more. The first parameter, `n_estimators`, shows the number of decision trees in the random forest. The second, `max_depth`, is the maximum depth of each tree. By picking three `n_estimators` values (10, 33, 100) and three `max_depth` values (2, 4, None), we allow for nine parameter combinations. For example, (`n_estimators` = 10, `max_depth` = 2) is a combination. The grid search tries all nine combinations and uses cross validation to choose the best one. This is done below.

What is cross validation?

Cross validation is a way to find out which parameter combination is the best. With 5-fold cross validation (the default) we divide the data into five equal-sized groups. Four of them are used as training data and the fifth is testing data. The score (in this case the root mean squared error) on the testing data is recorded. This is done for each split: that is, each fifth is taken out and used as testing data; then, the five root mean squared errors (from taking out the five fifths, one at a time) are averaged.

We now have root mean squared error value for each parameter combination, allowing us to choose the best combination.

```
np.random.seed(0)
gcv = GridSearchCV(estimator = RandomForestRegressor(), param_grid =
{"n_estimators": [10, 33, 100], "max_depth": [2, 4, None]}, \
                    scoring = "neg_root_mean_squared_error")
gcv.fit(train_df.values, train_target)

GridSearchCV(estimator=RandomForestRegressor(),
              param_grid={'max_depth': [2, 4, None],
```

```

        'n_estimators': [10, 33, 100]},
        scoring='neg_root_mean_squared_error')

print(gcv.best_estimator_)
print(gcv.best_params_)
print(gcv.best_score_)

RandomForestRegressor()
{'max_depth': None, 'n_estimators': 100}
-1.0851473707103607

gcv.cv_results_["mean_test_score"] # There are nine scores, one for
each of the 3 x 3 hyperparameter combinations

array([-2.27296628, -2.26369708, -2.26370023, -1.56063403, -
1.52732923,
       -1.52615891, -1.12289812, -1.0937049 , -1.08514737])

```

Why does `n_estimators` grow like 10, 33, 100 instead of like 1, 50, 100?

First of all, we can make it grow however we want. The question is why we chose 10, 33, 100. The idea here is that when you have no idea what is the appropriate number, you should pick guesses that grow exponentially. For example, suppose you are trying to guess the height of a particular tree. You look online and find that redwoods grow to 400 feet high, while bonsai trees may be as short as six inches. If you get three guesses, you could guess: one foot, 200 feet, 400 feet. These guesses would be linearly spaced out. However, the 200 foot guess feels wrong - it's still very tall. Most trees don't grow that tall. Instead, let's make a geometric progression:

```

list(map(round, [400**0, 400**0.25, 400**0.5, 400**0.75, 400*1]))

[1, 4, 20, 89, 400]

```

Now the guesses are four feet, 20 feet, and 89 feet. (We'll drop the initial 1 and the final 400.) That feels more plausible: the middle value, 20 feet, is representative of many actual trees in a way that the middle 200 foot guess wasn't.

2. Weekly graph question

In words, please summarize what this graph shows.

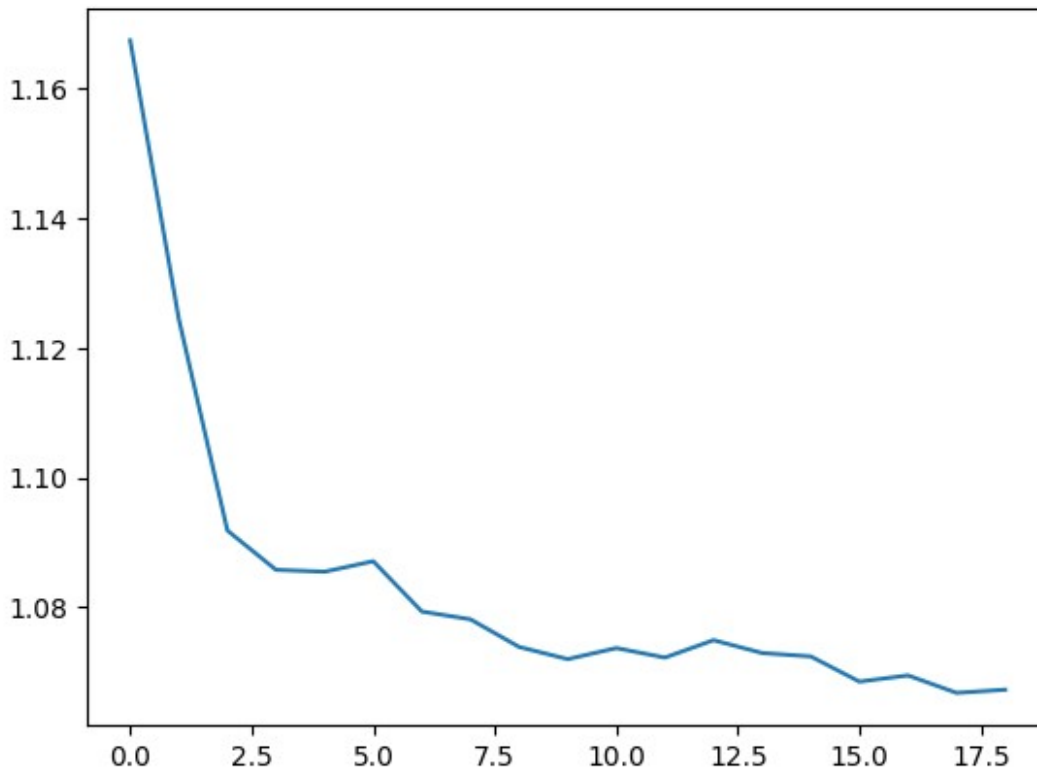
```

rmse_lst = list()
for x in range(round(num_points / 20), num_points, round(num_points /
20)):
    rf.fit(train_df.values[0:x,:], train_target[0:x])

rmse_lst.append(root_mean_squared_error(rf.predict(test_df.values),
test_target))

```

```
import matplotlib.pyplot as plt
plt.plot(rmse_lst)
[<matplotlib.lines.Line2D at 0x14172615e90>]
```



This graph shows the root mean squared error of a series of values. The error decreases over time

3. Homework

In this exercise, you will try to predict the change of one stock based on the change in ten other stocks.

```
def make_X_y(alpha = 0.5):
    df_hwk = pd.DataFrame()
    num_samples_hwk = 10000
    common_odd = np.random.normal(size = num_samples_hwk)
    common_even = np.random.normal(size = num_samples_hwk)
    common_three = np.random.normal(size = num_samples_hwk)
    for n in range(11):
        common = np.zeros(num_samples_hwk)
        if n % 2 == 0:
```

```

        common += common_even
    if n % 2 == 1:
        common += common_odd
    if n % 3 == 0:
        common += common_three
        common /= 2
    df_hwk["Stock_" + str(n)] = np.random.normal(size =
num_samples_hwk) + alpha * common
X = df_hwk.values[:, :-1]
y = df_hwk.values[:, -1]
return X, y

```

The above code creates an X with ten columns, corresponding to ten different stocks. The y is the eleventh stock. The stocks have some commonalities: the even numbered stocks are related, as are the odds. The stocks that are multiples of three are all related, too. Random forest should be able to sort through all this and decide that the eleventh stock is related to the odd numbered, non-divisible-by-three stocks, and predict accordingly. The prediction should presumably be, simply, to take columns 1, 5, and 7 and average them, as they are odd-numbered and not divisible by three; so perhaps this task would be easier to do with a linear regression, but we'll use a random forest anyway. The value "alpha" determines how important the common element is. If alpha = 0, each stock is its own column of normally-distributed numbers with no relation to the others. If alpha = 10, then the great majority of the variation in the stock values comes from the common elements.

Question:

Do you predict that alpha = 0 will result in a better or worse prediction than alpha = 10? Remember that the prediction must be performed on test data, so you'll need an X_train and y_train for training and a separate X_test and y_test for testing. I recommend using a correlation rather than a MSE or RMSE in this case, if you want to directly compare two different predictions, because the MSE and RMSE vary with the magnitude of the data as well as with the quality of the prediction.

After you make the prediction, try it out, and comment on what you learned.

I think it will be a worse prediction. since the stocks will be evaluated alone, then there will be no ability to predict the final stock based on the value of the other stocks.

Randomized Search

Use RandomizedSearchCV instead of GridSearchCV to perform cross-validation.

Scoring

Research the scoring parameters under "regression":

https://scikit-learn.org/dev/modules/model_evaluation.html#scoring-parameter

Choose at least one that you want to learn about, use it, and research it online. Write something about what you've learned about it.


```

from sklearn.model_selection import train_test_split
X1,y1 = make_X_y(alpha=0.0)
X2,y2 = make_X_y(alpha=10)
X,y = make_X_y()
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2)

from sklearn.model_selection import RandomizedSearchCV

np.random.seed(0)
RCV = RandomizedSearchCV(estimator = RandomForestRegressor(),
param_distributions={"n_estimators": [10, 33, 100], "max_depth": [2,
4, None]}, \
                        scoring = "neg_root_mean_squared_error")
RCV.fit(X_train, y_train)

C:\Users\thyde\AppData\Local\Packages\
PythonSoftwareFoundation.Python.3.11_qbz5n2kfra8p0\LocalCache\local-
packages\Python311\site-packages\sklearn\model_selection\
_search.py:317: UserWarning: The total space of parameters 9 is
smaller than n_iter=10. Running 9 iterations. For exhaustive searches,
use GridSearchCV.
  warnings.warn(

RandomizedSearchCV(estimator=RandomForestRegressor(),
                    param_distributions={'max_depth': [2, 4, None],
                                         'n_estimators': [10, 33,
100]}),
                    scoring='neg_root_mean_squared_error')

print(RCV.best_estimator_)
print(RCV.best_params_)
print(RCV.best_score_)

RandomForestRegressor(max_depth=4, n_estimators=10)
{'n_estimators': 10, 'max_depth': 4}
-1.0735782784155607

np.random.seed(0)
RCV = RandomizedSearchCV(estimator = RandomForestRegressor(),
param_distributions={"n_estimators": [10, 33, 100], "max_depth": [2,
4, None]}, \
                        scoring = "r2")
RCV.fit(X_train, y_train)

C:\Users\thyde\AppData\Local\Packages\
PythonSoftwareFoundation.Python.3.11_qbz5n2kfra8p0\LocalCache\local-
packages\Python311\site-packages\sklearn\model_selection\

```

```
_search.py:317: UserWarning: The total space of parameters 9 is
smaller than n_iter=10. Running 9 iterations. For exhaustive searches,
use GridSearchCV.
    warnings.warn(

RandomizedSearchCV(estimator=RandomForestRegressor(),
                    param_distributions={'max_depth': [2, 4, None],
                                         'n_estimators': [10, 33,
100]}),
                    scoring='r2')
```

I learned about R2. This is the coefficient of determination. R2 is an important model for determining if data is correlated, and how well the target value fits the independent variables of the model.

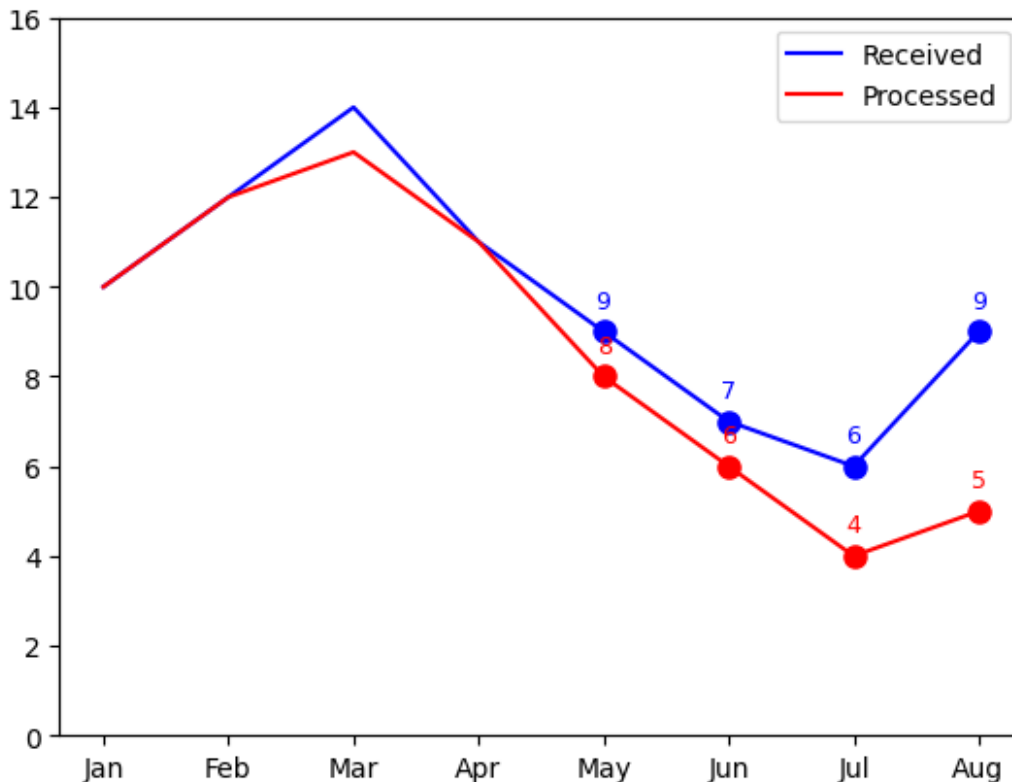
4. Storytelling With Data plot

Reproduce any graph of your choice in chapter six (p. 151-163) of the Storytelling With Data book as best you can. You do not have to get the exact data values right, just the overall look and feel.

```
received = [10,12,14,11,9,7,6,9]
Processed = [10,12,13,11,8,6,4,5]
Months = ["Jan", "Feb", "Mar", "Apr", "May", "Jun", "Jul", "Aug"]

plt.plot(Months, received, color="blue", label = "Received")
plt.plot(Months, Processed, color="red", label = "Processed")
plt.plot(Months[-4:], received[-4:], linestyle='', marker='o',
color='blue', markersize=8)
plt.plot(Months[-4:], Processed[-4:], linestyle='', marker='o',
color='red', markersize=8)
y_offset = 0.4
for x, y in zip(Months[-4:], received[-4:]):
    plt.text(x, y + y_offset, str(y), ha='center', va='bottom',
color='blue', fontsize=9)
for x, y in zip(Months[-4:], Processed[-4:]):
    plt.text(x, y + y_offset, str(y), ha='center', va='bottom',
color='red', fontsize=9)
plt.legend()
plt.ylim(0,16)

(0.0, 16.0)
```



Week 11 - Introduction to Modeling, part 2

1. Lesson - No lesson this week

2. Weekly graph question

```
import pandas as pd
import numpy as np
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import root_mean_squared_error
from sklearn.model_selection import GridSearchCV
from sklearn.metrics import make_scorer
```

The book names one of Vonnegut's rules as "keep it simple" and another as "have the guts to cut." Here is some data from the previous week's lesson. If you had to cut one of the two plots below, which would it be? Which seems more interesting or important? Explain. (Should "amount of training data used" or "number of estimators" be on the x-axis.)

I would cut the first graph. These models are good for predicting the hyperparameters that are most useful for machine learning. Since number of estimators is the important metric on that front, I think that it is the most important graph to preserve in this scenario.

```
np.random.seed(0)
num_points = 10000
feature_1a = np.random.random(size = num_points) * 3
feature_2a = np.random.random(size = num_points) * 3
feature_3a = np.random.random(size = num_points) * 3
train_target = (feature_1a - 2 * feature_2a) * feature_3a +
np.random.normal(size = num_points)
feature_1b = np.random.random(size = num_points) * 3
feature_2b = np.random.random(size = num_points) * 3
feature_3b = np.random.random(size = num_points) * 3
test_target = (feature_1b - 2 * feature_2b) * feature_3b +
np.random.normal(size = num_points)
train_df = pd.DataFrame({"f1": feature_1a, "f2": feature_2a, "f3":
feature_3a})
test_df = pd.DataFrame({"f1": feature_1b, "f2": feature_2b, "f3":
feature_3b})
rf = RandomForestRegressor()
rf.fit(train_df.values, train_target)

RandomForestRegressor()

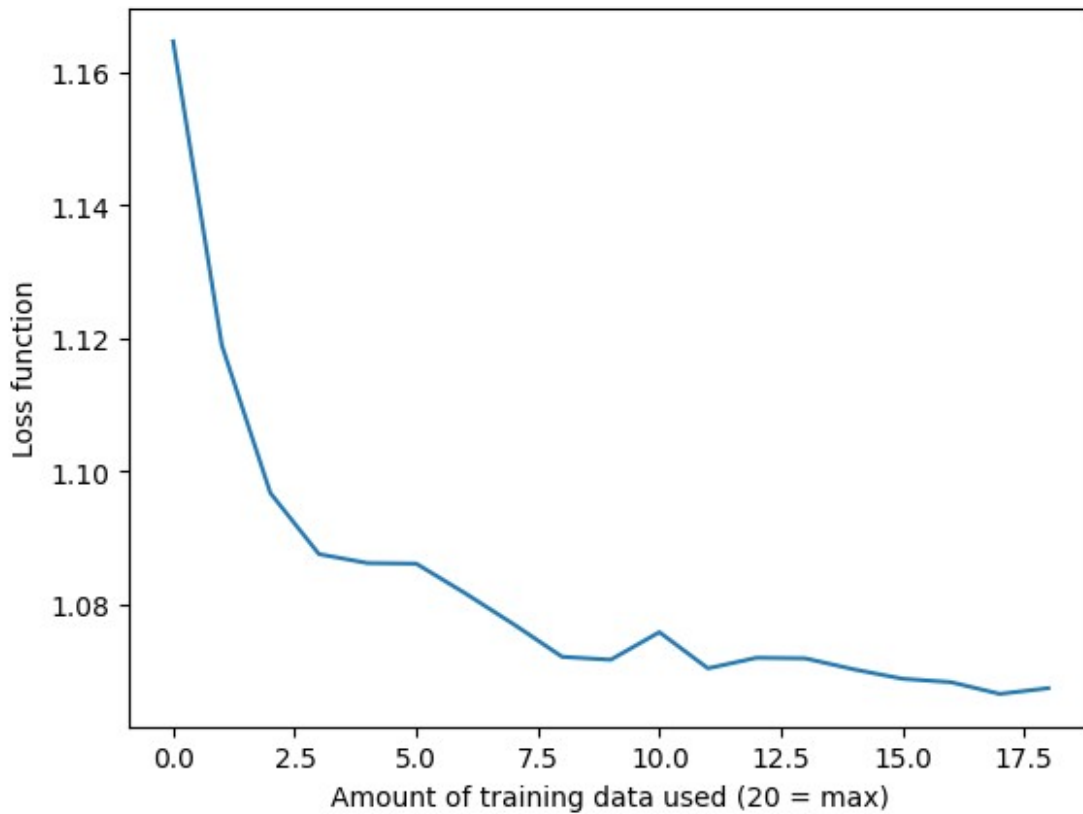
rmse_lst = list()
rf = RandomForestRegressor()
for x in range(round(num_points / 20), num_points, round(num_points /
20)):
    rf.fit(train_df.values[0:x,:], train_target[0:x])

rmse_lst.append(root_mean_squared_error(rf.predict(test_df.values),
test_target))

import matplotlib.pyplot as plt

plt.plot(rmse_lst)
plt.xlabel("Amount of training data used (20 = max)")
plt.ylabel("Loss function")

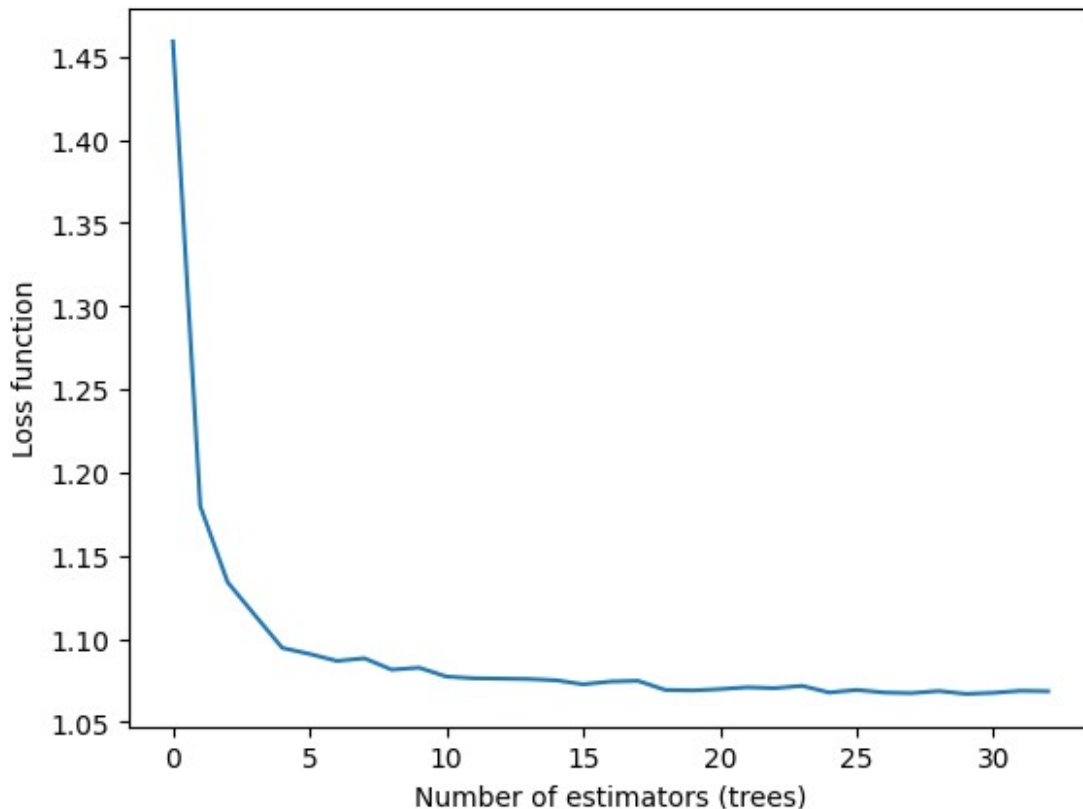
Text(0, 0.5, 'Loss function')
```



```
num_trees_lst = list()
for n_estimators in range(1, 100, 3):
    rf = RandomForestRegressor(n_estimators = n_estimators)
    rf.fit(train_df.values, train_target)

num_trees_lst.append(root_mean_squared_error(rf.predict(test_df.values
), test_target))

plt.plot(num_trees_lst)
plt.xlabel("Number of estimators (trees)")
plt.ylabel("Loss function")
Text(0, 0.5, 'Loss function')
```



3. Working on your datasets

This week, you will do the same types of exercises as last week, but you should use your chosen datasets that someone in your class found last semester. (They likely will not be the particular datasets that you found yourself.)

Here are some types of analysis you can do:

- Implement a random forest model.
- Perform cross-validation.
- Tune hyperparameters.
- Evaluate a performance metric.

If you like, you can try other types of models, too (beyond linear regression and random forest) although you will have many opportunities to do that next semester.

```
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score
from sklearn.model_selection import cross_val_score, validation_curve
from sklearn.metrics import root_mean_squared_error
from sklearn.metrics import mean_absolute_error
from sklearn.model_selection import RandomizedSearchCV
```

```

df_C02 = pd.read_csv(r"C:\Users\thyde\OneDrive\Documents\OMDS
Resources\DX 699 AI for Leaders\archive\owid-co2-data.csv")

numeric_df_C02 = df_C02.select_dtypes(include='number')

nona_numeric_df_C02 = numeric_df_C02.dropna()

clean_numeric_dfC02 =
nona_numeric_df_C02[['gdp', 'co2_growth_prct', 'co2_per_capita', 'co2_per
_gdp', 'temperature_change_from_co2']].copy()

y1 = clean_numeric_dfC02["temperature_change_from_co2"]
X1 = clean_numeric_dfC02.drop(columns=["temperature_change_from_co2"])

X1_train, X1_test, y1_train, y1_test = train_test_split(X1, y1,
test_size=0.2, random_state=42)

rf1 = RandomForestRegressor(n_estimators=200, random_state=42,
n_jobs=-1)
rf1.fit(X1_train, y1_train)

RandomForestRegressor(n_estimators=200, n_jobs=-1, random_state=42)

y_pred = rf1.predict(X1_test)

rmse = root_mean_squared_error(y1_test, y_pred)
mae = mean_absolute_error(y1_test, y_pred)
r2 = r2_score(y1_test, y_pred)

print(f"RMSE: {rmse}")
print(f"MAE: {mae}")
print(f"R²: {r2}")

RMSE: 0.033667285303897655
MAE: 0.0038743085106383007
R²: 0.7756144606904498

RCV1 = RandomizedSearchCV(estimator = RandomForestRegressor(),
param_distributions={"n_estimators": [10, 33, 100], "max_depth": [2,
4, None]}, scoring = "neg_root_mean_squared_error", random_state=42)
RCV_model1 = RCV1.fit(X1_train, y1_train)

C:\Users\thyde\AppData\Local\Packages\
PythonSoftwareFoundation.Python.3.11_qbz5n2kfra8p0\LocalCache\local-
packages\Python311\site-packages\sklearn\model_selection\
_search.py:317: UserWarning: The total space of parameters 9 is
smaller than n_iter=10. Running 9 iterations. For exhaustive searches,
use GridSearchCV.
  warnings.warn(

y_pred1 = RCV_model1.predict(X1_test)

```

```

rmse1 = root_mean_squared_error(y1_test, y_pred1)
mae1 = mean_absolute_error(y1_test, y_pred1)
r2_1 = r2_score(y1_test, y_pred1)

print(f"RMSE: {rmse1}")
print(f"MAE: {mae1}")
print(f"R2: {r2_1}")

RMSE: 0.03418987463131395
MAE: 0.004005425531914899
R2: 0.7685944967473058

GCV1 = GridSearchCV(estimator = RandomForestRegressor(),
param_grid={"n_estimators": [10, 33, 100], "max_depth": [2, 4, None]},
scoring = "neg_root_mean_squared_error")
GCV_model1 = GCV1.fit(X1_train, y1_train)

y2_pred = GCV_model1.predict(X1_test)

rmse2 = root_mean_squared_error(y1_test, y2_pred)
mae2 = mean_absolute_error(y1_test, y2_pred)
r2_2 = r2_score(y1_test, y2_pred)

print(f"RMSE: {rmse2}")
print(f"MAE: {mae2}")
print(f"R2: {r2_2}")

RMSE: 0.03328249524474258
MAE: 0.0037298936170212806
R2: 0.7807142433509228

import matplotlib.pyplot as plt

data = [
    [round(rmse, 4), round(rmse1, 4), round(rmse2, 4)],
    [round(mae, 4), round(mae1, 4), round(mae2, 4)],
    [round(r2, 4), round(r2_1, 4), round(r2_2, 4)]
]

fig, ax = plt.subplots(figsize=(8, 3))
ax.axis("off")

table = ax.table(
    cellText=data,
    rowLabels=["RMSE", "MAE", "R2"],
    colLabels=["Baseline RF Model", "Random Search", "Grid Search"],
    cellLoc="center",
    loc="center"
)

table.auto_set_font_size(False)

```



```

table.set_fontsize(12)
table.scale(1.2, 1.8)

plt.title("FIG. 6: CO2 Emissions Per Capita: Random Forest Model Comparison", fontsize=14, fontweight="bold", pad=20)
plt.tight_layout()
plt.show()

```

FIG. 6: CO2 Emissions Per Capita: Random Forest Model Comparison

	Baseline RF Model	Random Search	Grid Search
RMSE	0.0337	0.0342	0.0333
MAE	0.0039	0.004	0.0037
R ²	0.7756	0.7686	0.7807

```

df_agr = pd.read_csv(r"C:\Users\thyde\OneDrive\Documents\OMDS Resources\DX 699 AI for Leaders\archive\climate_change_impact_on_agriculture_2024 - climate_change_impact_on_agriculture_2024.csv")
df_usa_agr = df_agr[df_agr["Country"] == "USA"]
df_usa_agr_numeric = df_usa_agr.select_dtypes(include="number")

df_usa_agr_numeric.columns

Index(['Year', 'Average Temperature (C)', 'Total Precipitation (mm)',
      'CO2 Emissions (MT)', 'Crop Yield (MT per HA)',
      'Extreme Weather Events', 'Irrigation Access %',
      'Pesticide Use (KG per HA)', 'Fertilizer Use (KG per HA)',
      'Soil Health Index', 'Economic Impact (Million USD)'],
      dtype='object')

y2 = df_usa_agr_numeric["Crop Yield (MT per HA)"]
X2 = df_usa_agr_numeric.drop(columns=["Crop Yield (MT per HA)"])

X2_train, X2_test, y2_train, y2_test = train_test_split(X2, y2,
test_size=0.2, random_state=42)

rf2 = RandomForestRegressor(n_estimators=200, random_state=42,
n_jobs=-1)

rf2.fit(X2_train, y2_train)

RandomForestRegressor(n_estimators=200, n_jobs=-1, random_state=42)

```

```
y3_pred = rf2.predict(X2_test)
```

```
rmse3 = root_mean_squared_error(y2_test, y3_pred)
```

```
mae3 = mean_absolute_error(y2_test, y3_pred)
```

```
r2_3 = r2_score(y2_test, y3_pred)
```

```
print(f"RMSE: {rmse3}")
```

```
print(f"MAE: {mae3}")
```

```
print(f"R2: {r2_3}")
```

```
RMSE: 0.6412923605803893
```

```
MAE: 0.5306150483091786
```

```
R2: 0.5595671872681873
```

```
RCV2 = RandomizedSearchCV(estimator = RandomForestRegressor(),
```

```
param_distributions={"n_estimators": [10, 33, 100], "max_depth": [2, 4, None]}, scoring = "neg_root_mean_squared_error", random_state=42)
```

```
RCV_model2 = RCV2.fit(X2_train, y2_train)
```

```
C:\Users\thyde\AppData\Local\Packages\
```

```
PythonSoftwareFoundation.Python.3.11_qbz5n2kfra8p0\LocalCache\local-packages\Python311\site-packages\sklearn\model_selection\
```

```
_search.py:317: UserWarning: The total space of parameters 9 is smaller than n_iter=10. Running 9 iterations. For exhaustive searches, use GridSearchCV.
```

```
warnings.warn(
```

```
y4_pred = RCV_model2.predict(X2_test)
```

```
rmse4 = root_mean_squared_error(y2_test, y4_pred)
```

```
mae4 = mean_absolute_error(y2_test, y4_pred)
```

```
r2_4 = r2_score(y2_test, y4_pred)
```

```
print(f"RMSE: {rmse4}")
```

```
print(f"MAE: {mae4}")
```

```
print(f"R2: {r2_4}")
```

```
RMSE: 0.6467392580855548
```

```
MAE: 0.533227836393745
```

```
R2: 0.5520536704727712
```

```
GCV2 = GridSearchCV(estimator = RandomForestRegressor(),
```

```
param_grid={"n_estimators": [10, 33, 100], "max_depth": [2, 4, None]}, scoring = "neg_root_mean_squared_error")
```

```
GCV_model2 = GCV2.fit(X2_train, y2_train)
```

```
y5_pred = GCV_model2.predict(X2_test)
```

```
rmse5 = root_mean_squared_error(y2_test, y5_pred)
```

```
mae5 = mean_absolute_error(y2_test, y5_pred)
```

```
r2_5 = r2_score(y2_test, y5_pred)
```

```
print(f"RMSE: {rmse5}")
```

```

print(f"MAE:  {mae5}")
print(f"R2:  {r2_5}")

RMSE: 0.6407134053671347
MAE: 0.5277132929869252
R2: 0.5603620689368556

import matplotlib.pyplot as plt

data = [
    [round(rmse3, 4), round(rmse4, 4), round(rmse5, 4)],
    [round(mae3, 4), round(mae4, 4), round(mae5, 4)],
    [round(r2_3, 4), round(r2_4, 4), round(r2_5, 4)]
]

fig, ax = plt.subplots(figsize=(8, 3))
ax.axis("off")

table = ax.table(
    cellText=data,
    rowLabels=["RMSE", "MAE", "R2"],
    colLabels=["Baseline RF Model", "Random Search", "Grid Search"],
    cellLoc="center",
    loc="center"
)

table.auto_set_font_size(False)
table.set_fontsize(12)
table.scale(1.2, 1.8)

plt.title("FIG. 7: Agriculture vs Climate Change: Random Forest Model Comparison", fontsize=14, fontweight="bold", pad=20)
plt.tight_layout()
plt.show()

```

FIG. 7: Agriculture vs Climate Change: Random Forest Model Comparison

	Baseline RF Model	Random Search	Grid Search
RMSE	0.6413	0.6467	0.6407
MAE	0.5306	0.5332	0.5277
R ²	0.5596	0.5521	0.5604

4. Storytelling With Data plot

Reproduce any graph of your choice in chapter seven (p. 165-185) of the Storytelling With Data book as best you can. You do not have to get the exact data values right, just the overall look and feel.

```
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import numpy as np

years_actual = [2006, 2007, 2008, 2009, 2010, 2011, 2012, 2013, 2014]
sales_actual = [50, 54, 58, 62, 76, 83, 90, 98, 108]

years_forecast = [2014, 2015, 2016, 2017, 2018]
sales_forecast = [108, 119, 131, 144, 158]

fig, ax = plt.subplots(figsize=(12, 6))

# Plot lines
ax.plot(years_actual, sales_actual, color='steelblue', linewidth=2.5,
        marker='o', markersize=4)
ax.plot(years_forecast, sales_forecast, color='steelblue',
        linewidth=2, linestyle='--',
        marker='o', markersize=6, markerfacecolor='white',
        markeredgewidth=2)

# Forecast shaded region
ax.axvspan(2014.5, 2018.5, alpha=0.07, color='steelblue')

# Data labels on forecast points
for x, y in zip(years_forecast[1:], sales_forecast[1:]):
    ax.annotate(f'${y}', (x, y), textcoords='offset points',
               xytext=(0, 10),
               ha='center', fontsize=10, color='steelblue')

# Annotations
ax.text(2006, 195, '2006–09: annual\nsales growth of 7–8%',
        fontsize=8.5, color='steelblue')
ax.text(2009.7, 195, '2010: more marked\nincrease of 22%\nsales YoY',
        fontsize=8.5, color='steelblue')
ax.text(2011, 195, '2011–14: steady\ngrowth of 8–9%', fontsize=8.5,
        color='steelblue')
ax.text(2014.7, 195, '2015 & beyond: assumed\n10% YoY increase in\nsales*',
        fontsize=8.5,
        color='steelblue', fontweight='bold')

# ACTUAL / FORECAST labels
ax.text(2009.5, -18, 'ACTUAL', ha='center', fontsize=9, color='gray',
        transform=ax.transData)
```

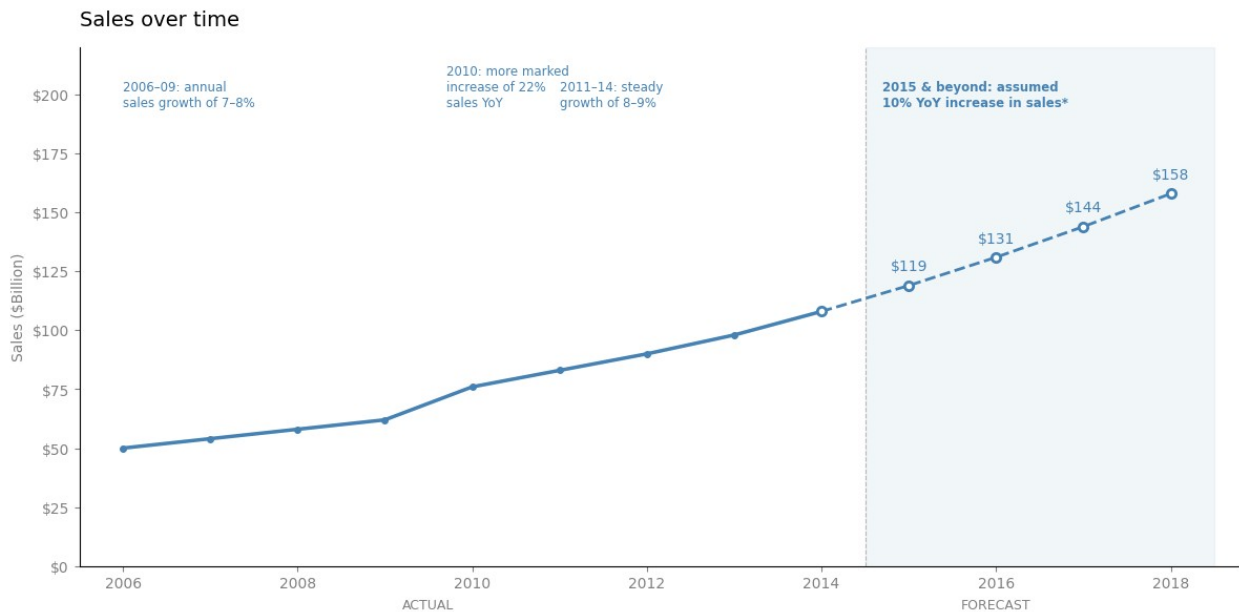
```

ax.text(2016, -18, 'FORECAST', ha='center', fontsize=9, color='gray',
transform=ax.transData)
ax.axvline(x=2014.5, color='gray', linestyle='--', linewidth=0.8,
alpha=0.5)

ax.set_ylim(0, 220)
ax.set_xlim(2005.5, 2018.8)
ax.set_ylabel('Sales ($Billion)', fontsize=10, color='gray')
ax.yaxis.set_major_formatter(plt.FuncFormatter(lambda x, _: f'$
{int(x)}'))
ax.tick_params(colors='gray')
ax.set_title('Sales over time', fontsize=14, loc='left', pad=15)
ax.spines[['top', 'right']].set_visible(False)

plt.tight_layout()
plt.savefig('sales_over_time.jpeg', dpi=150, bbox_inches='tight')
plt.show()

```



Week 12 - Milestone Two week

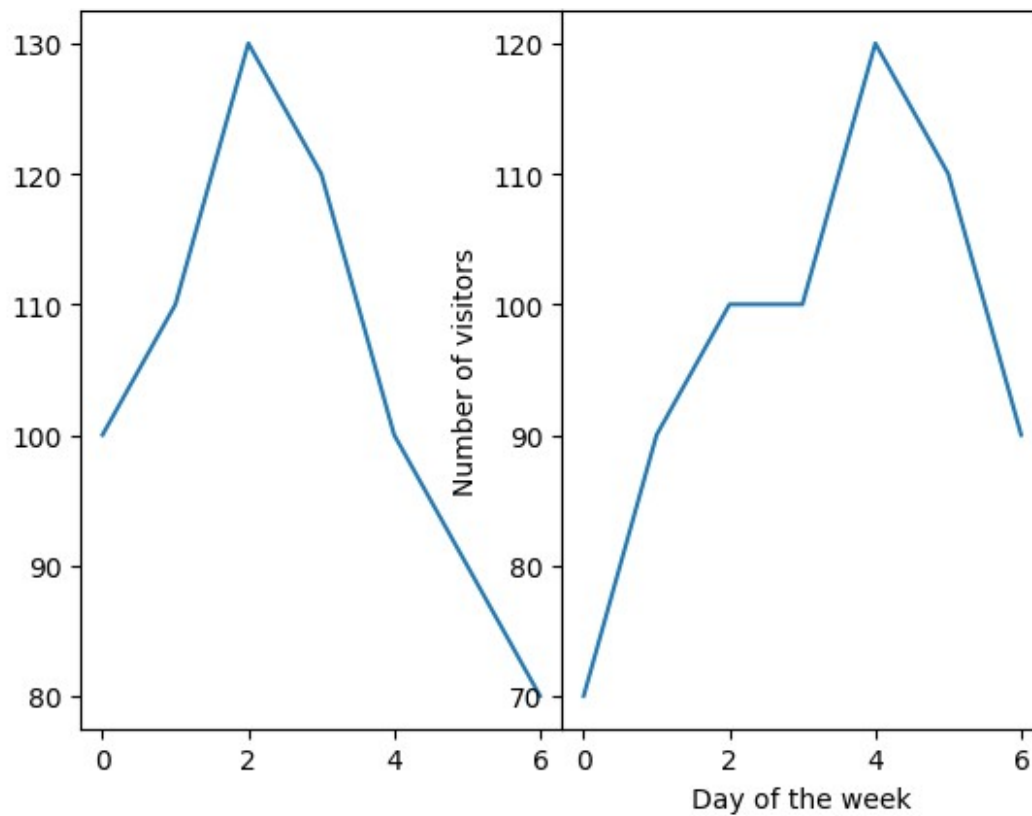
1. Lesson: no lesson this week. Milestone Two is due!

2. Weekly graph question

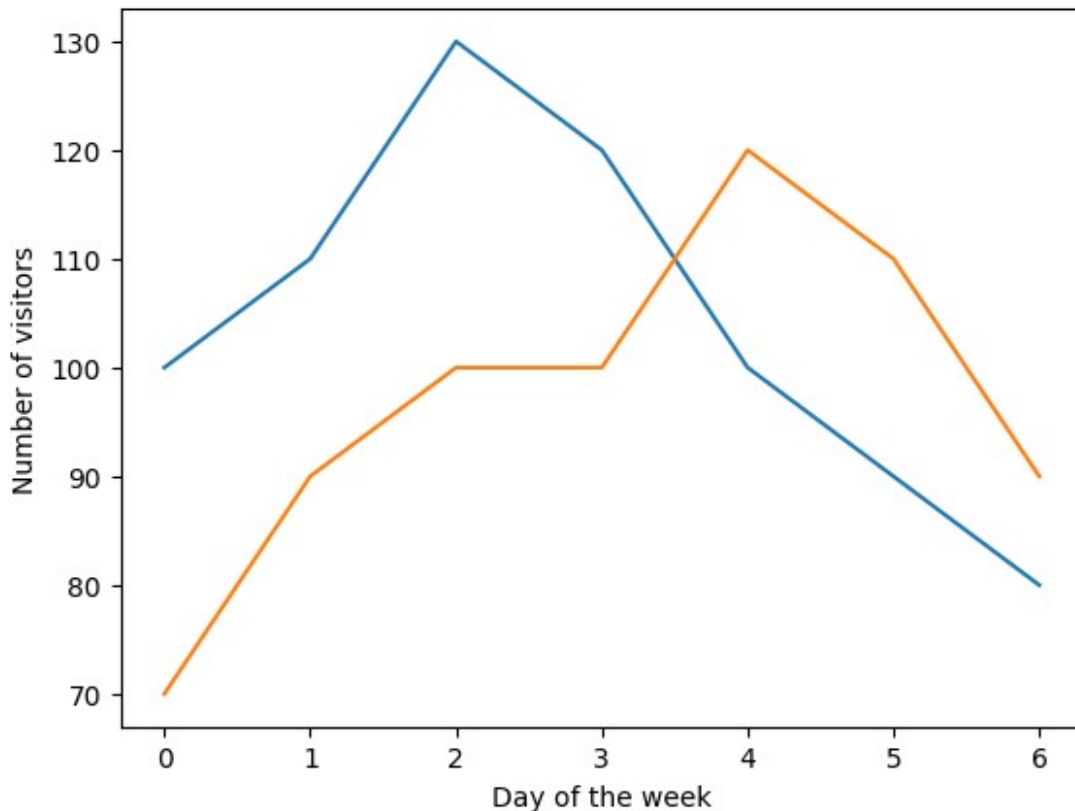
Suppose you plot the number visitors to a museum over the seven days of a week; then again the following week. Can you suggest the advantages and disadvantages of the following two ways of presenting the data?

```
import numpy as np
import seaborn as sns
import pandas as pd
import matplotlib.pyplot as plt

np.random.seed(0)
time_series_0 = np.array([100, 110, 130, 120, 100, 90, 80])
time_series_1 = np.array([70, 90, 100, 100, 120, 110, 90])
fig, axs = plt.subplots(1, 2)
axs[0].plot(time_series_0)
axs[1].plot(time_series_1)
plt.subplots_adjust(wspace = 0)
plt.xlabel("Day of the week")
plt.ylabel("Number of visitors")
Text(0, 0.5, 'Number of visitors')
```



```
plt.plot(time_series_0)
plt.plot(time_series_1)
plt.xlabel("Day of the week")
plt.ylabel("Number of visitors")
Text(0, 0.5, 'Number of visitors')
```



The top graph is less confusing without a legend. It shows the two days distinctly. The issue with the top graph is that the graphs are not as easily compared. The x axis also looks clunky, as it does not show every data point. The bottom graph better compares the two datasets, but is harder to interpret without a legend. A legend would be needed in order to make that the best choice.

3. Working on your datasets

This week, you can work on Milestone Two rather than having a particular exercise about working on your datasets

4. Storytelling With Data plot

Reproduce any graph of your choice in p. 187-205 of the Storytelling With Data book as best you can. (Chapter Eight). You do not have to get the exact data values right, just the overall look and feel.

```
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.ticker as mtick
years = np.arange(2008, 2015)
```



```

A = np.array([350, 400, 420, 430, 410, 300, 280])
B = np.array([330, 380, 405, 415, 395, 290, 270])
C = np.array([50, 80, 100, 150, 200, 220, 230])
D = np.array([70, 95, 130, 190, 240, 210, 205])
E = np.array([20, 40, 60, 110, 180, 140, 250])

series = [A, B, C, D, E]
labels = ['A', 'B', 'C', 'D', 'E']

fig, ax = plt.subplots(figsize=(8, 3.2))

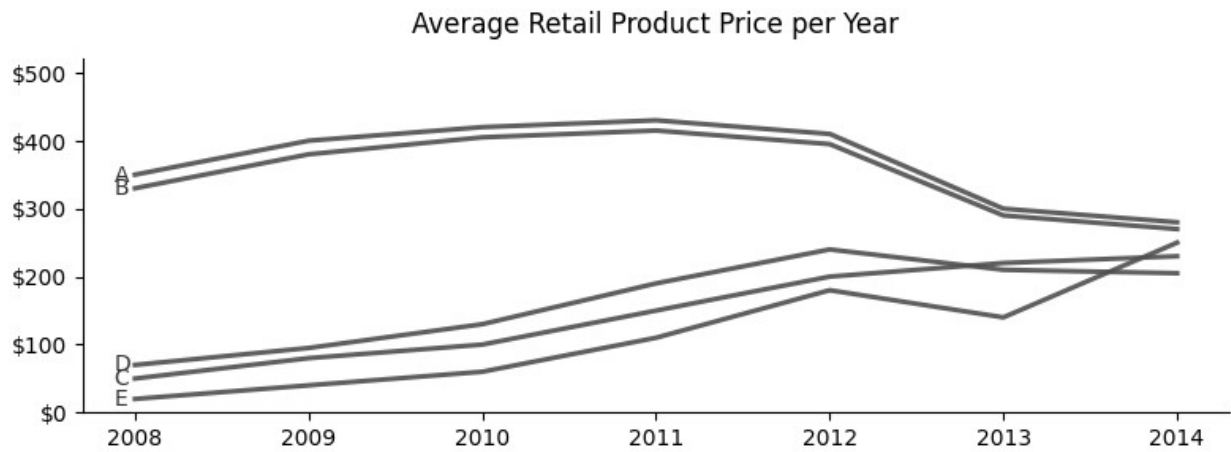
line_colors = ['#4f4f4f'] * len(series)
line_width = 2.2
alpha = 0.9

for y, color in zip(series, line_colors):
    ax.plot(years, y, color=color, linewidth=line_width, alpha=alpha,
            zorder=2)

x_label_pos = years[0] - 0.12
for y_vals, lab in zip(series, labels):
    y0 = y_vals[0]
    ax.text(x_label_pos, y0, lab, va='center', ha='left', fontsize=10,
            color='#333333')
ax.set_title('Average Retail Product Price per Year', fontsize=12,
            pad=12)
ax.set_xticks(years)
ax.set_xlim(years[0] - 0.3, years[-1] + 0.3)
ax.yaxis.set_major_formatter(mtick.StrMethodFormatter('${x:,.0f}'))
ax.set_ylim(0, 520)
ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)

plt.tight_layout(rect=[0, 0.04, 1, 1])
plt.show()

```



```
!nbmerge OMDS-ModB2-Week8-Hyde-Thomas.ipynb OMDS-ModB2-Week9-Hyde-Thomas.ipynb OMDS-ModB2-Week10-Hyde-Thomas.ipynb OMDS-ModB2-Week11-Hyde-Thomas.ipynb OMDS-ModB2-Week12-Hyde-Thomas.ipynb > Hyde_MergedHW_Weeks8_to_12.ipynb
```